과정 1

RAG 기반 Multi-AI Agent 만들기

하나의 평가셋으로 지식에 접근하는 다섯 가지 방식을 직접 만들어 비교하고, 금융 4조건으로 운영할 구조를 고릅니다.

2026.10.27(화)-28(수) · 14시간 | 코스콤 중급 이상 개발자 | 강의 + 1인 1PC 실습

1일차 오전
Part 1
기준선과 하이브리드 RAG

1일차 오후
Part 2
품질 측정, 에이전틱 검색, llm-wiki

2일차 오전
Part 3
도구 연결과 멀티 에이전트

2일차 오후
Part 4
비용, 보안, 구조 선택

SESSION 01 / 14

1-1

1일차
09:00-10:00

강의 **30분**
LangGraph 시연 **10분**
실습 **20분**

RAG 기반 Multi-AI Agent 만들기

RAG의 현재 위상, 통째 넣기 기준선

이번 시간에는 RAG에 관한 최근 논의를 정리하고, 규정집 전체를 프롬프트에 넣었을 때의 점수를 기준선으로 측정한다.

학습 목표

1. **기존 RAG 방식의 한계와 최근 대안을 설명할 수 있다**
2. **규정집 전체를 넣는 방식으로 평가셋을 실행하고, 점수와 질문당 토큰, 비용을 기록한다**

결과물 · 기준선 점수표

이틀 동안 다섯 가지 방식을 비교한다

같은 평가셋과 같은 채점 방법으로 다섯 가지 방식을 차례로 측정한다. 조건이 같으므로 점수 차이는 방식의 차이로 볼 수 있다.

순서	방식	실행 주체	언제
0	통째 넣기 기준선	Claude Code 헤드리스	1일차 09시
1	하이브리드 RAG	LangGraph	1일차 11시
2	에이전틱 검색	LangGraph	1일차 15시
3	llm-wiki	LangGraph	1일차 16시
4	멀티 에이전트	LangGraph	2일차 10시

각 방식은 금융 4조건으로 평가한다

점수 하나로는 실제 업무에 쓸 수 있는지 판단하기 어렵다. 그래서 출처, 권한, 현행성, 측정의 네 가지 조건을 함께 본다.

조건	판정 질문	측정 방법
출처	근거 조항(문서·조·시행일)을 제시하는가	인용 누락 건수
권한	권한 밖 문서가 답에 섞이지 않는가	권한 시나리오 위반 건수
현행성	개정 전 조항이 답에 섞이지 않는가	함정(개정) 문항 오답 건수
측정	품질과 비용을 수치로 보일 수 있는가	점수, 질문당 토큰, 지연

[1-1] RAG의 현재 위상, 통째 넣기 기준선

RAG의 현재 위상

기존 RAG 방식의 한계

기존 방식(naive RAG)은 질문으로 한 번 검색하고 바로 답을 만든다. 첫 검색에서 맞는 조항을 찾지 못하면 다시 찾는 단계가 없다.



과정 지도 | RAG의 현재 위상 | 금융 4조건 | LangGraph 시연 | 기준선 실습

"RAG는 필요 없다"는 주장

RAG를 쓰지 않겠다는 주장은 주로 코드베이스를 다루는 코딩 에이전트 쪽에서 나왔다.

인용

for autonomous coding agents, RAG is a MASSIVE DISTRACTION

자율 코딩 에이전트에게 RAG는 오히려 방해가 된다

rag_research_report.md [43] — https://pashpashpash.substack.com/p/why-i-no-longer-recommend-rag-for

인용

No RAG. No embeddings. No vector databases.

RAG도, 임베딩도, 벡터 DB도 쓰지 않는다

rag_research_report.md [44] — https://cline.bot/blog/why-cline-doesnt-index-your-codebase-and-why-thats-a-good-thing

반론: 검색 방식을 바꾸자는 입장

반론 쪽은 검색은 계속 필요하고, 에이전트가 필요할 때 여러 번 검색하는 방식으로 바꾸면 된다고 본다.

인용

The question isn't whether to use retrieval, but how to retrieve effectively.

검색을 쓸지 말지가 아니라, 어떻게 잘 검색할지가 문제다

rag_research_report.md [45] — https://hamel.dev/blog/posts/evals-faq/is-rag-dead.html

인용

RAG is dead, long live agentic retrieval

기존 RAG 대신 에이전트가 검색하는 방식(agentic retrieval)으로 간다

rag_research_report.md [46] — https://www.llamaindex.ai/blog/rag-is-dead-long-live-agentic-retrieval

```
Cursor 자체 측정 (2025-11): grep에 임베딩 검색을 더하면 정확도 +12.5%
```

학계는 RAG를 어떻게 보나

2025~26년 서베이들은 RAG를 따로 떨어진 기법보다는 에이전트 구조나 컨텍스트 엔지니어링의 일부로 다룬다.

원문 요약

- ✓ Singh 외 [1]: 에이전트 수, 제어 구조, 자율성, 지식 표현의 네 기준으로 agentic RAG를 분류
- ✓ SoK (2026-03) [3]: agentic RAG의 반복 과정을 수학적 모델(POMDP)로 정리. 고정된 평가셋만으로는 제대로 측정하기 어렵다고 봄
- ✓ Context Engineering 서베이 (2025-07) [4]: RAG를 컨텍스트 엔지니어링(모델에 넣을 지시문, 메모리, 도구 결과, 검색 결과 전체를 설계하는 일)의 한 방법으로 봄

rag_research_report.md §2.1 [1][3][4] · 원문 요약

긴 컨텍스트와 RAG를 비교한 연구

긴 컨텍스트와 RAG 중 항상 나은 쪽은 없었다. 입력이 길어질수록 모델이 문맥 안에서 정보를 찾는 성능도 떨어졌다.

원문 요약

- ✓ LaRA (2,326개 사례) [7]: 모델, 과제, 문서 조각의 특성에 따라 결과가 달라짐. 어느 한쪽이 항상 낫지 않음
- ✓ Chroma Context Rot (2025-07) [8]: 18개 모델 모두 입력이 길어질수록 성능이 불안정해짐
- ✓ Sewon Min 외 (2026-07) [9]: 100만 토큰 규모에서는 주의가 분산되어(attention dilution) 문맥 안에서 찾는 성능이 크게 떨어짐

rag_research_report.md §2.2 [7][8][9] · 원문 요약

규정집 전체를 넣어 기준선을 잰다

Anthropic은 문서 전체가 20만 토큰보다 작으면 검색 없이 전부 넣으라고 권한다. 실습 규정집도 이 크기 안에 들어가므로, 검색을 만들기 전에 이 방식의 점수를 먼저 잰다.

원문 요약

지식베이스가 20만 토큰 미만이면 RAG 없이 전부 프롬프트에 넣고 프롬프트 캐싱을 쓰라고 권장

rag_research_report.md §3.1 [32] — https://www.anthropic.com/news/contextual-retrieval · 원문 요약

```
실습 규정집: 문서 15~25개, 약 10만~15만 토큰 (데이터 확정 후 실측값으로 교체)
```

업계의 세 가지 흐름

업계의 흐름은 컨텍스트 엔지니어링, 에이전틱 검색, 지식 컴파일 세 가지로 볼 수 있다. 이 과정의 실습도 이 순서로 다룬다.

흐름	내용	대표 사례	이번 과정
컨텍스트 엔지니어링	모델에 넣을 맥락 전체를 설계한다. RAG는 그중 한 부분	Anthropic [34], Simon Willison [51]	전 과정
에이전틱 검색	필요할 때 도구로 찾아 읽는다	Claude Code [41], Cursor [48]	2-3
지식 컴파일	원자료를 미리 위키로 정리해 두고 질의한다	Karpathy [66], LangChain [58], Pinecone [62]	과제 1

금융분야 AI 가이드라인의 보조수단성 원칙

가이드라인은 최종 결정과 책임이 직원에게 있다고 정한다. 직원이 답을 확인할 수 있도록 근거가 된 조항을 함께 보여 줘야 한다.

원문 요약

금융분야 AI 가이드라인 (2026-06-22 시행, 자율규제)
7대 원칙 중 "보조수단성": 최종 결정과 책임은 직원에게 있음

rag_research_report.md §7.2 [107] — https://www.fsc.go.kr/no010101/87142 · 원문 요약

(선택) 코스콤의 에이전트 도입

코스콤 AX 추진단의 초기 성과는 개발 생산성 쪽에서 나왔다. 보도에는 사내 문서 검색(RAG)에 관한 언급이 없다.

원문 요약

- ✓ AX 추진단 출범 (2026-09-16)
- ✓ 에이전트로 C 코드를 Java로 전환해 개발 공수 30% 이상 절감
- ✓ 사내 챗봇으로 관련 업무 30% 감소

rag_research_report.md §7.3 [111] — https://www.newspim.com/news/view/20260918000954 · 원문 요약

[1-1] RAG의 현재 위상, 통째 넣기 기준선

LangGraph 시연

LangGraph 그래프의 세 가지 구성 요소

이 과정에서 만드는 방식은 모두 LangGraph 그래프다. 그래프는 상태(State)를 받아 일부 값을 바꿔 돌려주는 노드(Node)를 엣지(Edge)로 이은 것이다.

Python

```python
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):
    question: str
    answer: str

def answer(state: State) -> dict:
    reply = llm.invoke(state["question"])   # llm: 고정 모델
    return {"answer": reply.content}

builder = StateGraph(State)
builder.add_node("answer", answer)
builder.add_edge(START, "answer")
builder.add_edge("answer", END)
graph = builder.compile()
graph.invoke({"question": "시세 전송 지연 시 참가사 통보 기준은?"})
```

```
State  노드 사이에 주고받는 데이터 (여기서는 question, answer)
Node   State를 받아 바꿀 값을 돌려주는 함수 (answer)
Edge   다음에 실행할 노드 (START → answer → END)
```

조건부 엣지로 분기를 만든다

조건부 엣지는 State 값을 보고 다음 노드를 고른다. 이후 세션의 답변 거부(2-2), 검색 반복(2-3), 승인 대기(4-2)를 모두 이 기능으로 만든다.

Python

```
def route(state: State) -> str:
    return "refuse" if not state["sufficient"] else "finish"

builder.add_conditional_edges("generate", route, {"refuse": "refuse",
```

```
2-2  근거가 부족하면 → "문서에 없음" 노드로
2-3  도구를 더 호출해야 하면 → 도구 노드로
되돌아감 (반복)
4-2  비밀 등급 문서를 인용하면 → 사람 승인 대기
(interrupt)
```

[1-1] RAG의 현재 위상, 통째 넣기 기준선

실습: 통째 넣기 기준선

실습 ① 질문 1건으로 먼저 확인하기

Claude Code에 규정집 파일 하나만 읽게 하고 질문 1건을 실행해, 답에 출처가 붙는지 확인한다.

Terminal

```
git checkout s1-1-start
```

```
data/regs_all.md 전체를 읽고, 그
내용만 근거로 다음 질문에 답해줘.
다른 파일은 열지 마. 답에는 문서명·조항·
시행일을 인용해.
질문: 시세 전송 지연 시 참가사 통보
기준은?
```

확인할 것

- 답에 문서명, 조항, 시행일이 붙었는가
- data/regs_all.md 외에 다른 파일을 열지 않았는가

실습 ② 평가셋 30건을 한 번에 실행하기

30건을 하나씩 입력하지 않고 스크립트로 실행해야 이후 다른 방식과 같은 조건으로 비교할 수 있다. 헤드리스 모드(claude -p)는 Claude Code를 대화 화면 없이 명령 한 줄로 실행하는 방식이다.

```
scripts/run_claude_baseline.sh 를 만들어줘.
- eval/questions.jsonl의 각 질문에 대해 claude
-p를 호출
- 도구는 Read만 허용, 읽을 파일은 data/regs_all.md
하나
- --output-format json 결과에서 답변·토큰·비용을
추출해
  eval/results/baseline_r1.jsonl 형식
(CLAUDE.md 참고)으로 저장
- --model은 CLAUDE.md의 고정 모델 사용
```

```
실행 전에 스크립트를 검수한다
- 허용 도구가 Read 하나인가
- --model이 CLAUDE.md의 고정 모델인가
```

확인 기준과 막힐 때

결과 파일에 30건의 답과 질문당 토큰, 비용이 기록되어 있으면 이번 실습은 성공이다.

```
확인 기준
- baseline_r1.jsonl에 30건의 답이 있다
- 각 건에 질문당 토큰과 비용이 기록되어 있다

막힐 때
- 헤드리스 실행 오류 → solutions/scripts/run_claude_baseline.sh 사용
- 시간 부족          → 강사가 제공하는 baseline_r1.jsonl 사용
```

다음 시간: 문서 적재와 평가 러너

다음 시간에는 규정을 조항 단위로 나눠 저장하고, 모든 방식을 같은 기준으로 실행하는 평가 러너를 만든다.

```
1-2 문서 적재, 평가셋 (10:00–11:00)
결과물: src/ingest.py · src/run_eval.py
```

과정 지도 | RAG의 현재 위상 | 금융 4조건 | LangGraph 시연 | 기준선 실습

SESSION 02 / 14

1-2

1일차
10:00-11:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

문서 적재, 평가셋

이번 시간에는 규정 문서를 조항 단위로 나눠 저장하고, 모든 방식을 같은 기준으로 실행하는 평가 러너를 만든다.

학습 목표

1. **조항 단위 적재와 메타데이터가 이후 모든 방식의 기반이 되는 이유를 설명할 수 있다**
2. **모든 방식을 같은 기준으로 실행하는 평가 러너를 만든다**

결과물 · src/ingest.py, src/run_eval.py

규정 문서를 조항 단위로 나눈다

규정 문서에는 이미 조와 항이라는 구조가 있다. 고정 길이로 자르는 것보다 이 구조대로 나누는 편이 낫다.

고정 길이로 자르기 (500자)

… 제11조 ③ 통보는
제12조 (지연 기준)

① 시세 전송이 …
② 제1항에 따른 …

조항 경계가 조각 중간에 걸린다

조항 단위로 나누기

제11조 (통보 방법)

제12조 (지연 기준)
① 시세 전송이 …
② 제1항에 따른 …

조각 1개가 인용 단위 1개가 된다

조각에 문서 맥락을 붙이는 방법

Anthropic의 Contextual Retrieval은 조각마다 문서 맥락을 덧붙여 검색 실패를 줄였다. 이번 과정에서는 설명만 하고 실습하지 않는다.

원문 요약

조각에 맥락을 덧붙이고 BM25(키워드 검색)와 리랭크(검색 결과 재정렬)를 함께 쓰면, 상위 20건 안에 정답이 없는 비율(검색 실패율)이 67% 감소

rag_research_report.md §3.1 [32] — https://www.anthropic.com/news/contextual-retrieval · 원문 요약

이번 실습에서는 문서명, 조항 제목, 시행일 메타데이터로 이 역할을 대신한다

메타데이터 네 가지와 쓰이는 곳

조항, 시행일, 보안 등급, 개정 이력 네 가지 메타데이터를 붙인다. 각각 인용, 현행성, 권한을 다루는 세션에서 쓴다.

메타데이터	예시	쓰이는 곳
조항 (doc, article)	시세정보전송규정 제12조	2-2 출처 인용
시행일 (effective_date)	2026-03-01	4-3 현행 조항 필터
보안 등급 (security_level)	일반 / 대외비 / 비밀	4-2 권한 필터
개정 이력 (revision)	개정 전후본 연결	2-1 함정 문항, 4-3

평가셋 30건의 문항 유형

평가셋 30건은 여섯 유형으로 나뉜다. 유형별 점수를 보면 어떤 방식이 어떤 질문에 강하고 약한지 알 수 있다.

유형	건수	보려는 것
단일 사실	10	기본 정확도
교차 참조 (다른 조항을 따라가야 답이 나옴)	5	에이전틱 검색, 위키가 잘하는지
종합 (여러 문서를 함께 봐야 함)	5	위키가 잘하는지
함정: 개정 전 조항	4	현행성
함정: 표현 변형 (같은 뜻, 다른 단어)	3	벡터 검색과 BM25의 차이
권한 밖	3	권한 (사용자 역할 지정)
합계	**30**	

별도 문항 세트: 문서 밖 질문 10건(2-2), 내부와 외부 결합 10건(3-3), DART 단독 10건(3-1), 인젝션과 권한 공격 5건(4-2)

[1-2] 문서 적재, 평가셋

실습: 적재와 평가 러너

실습 ① 조항 단위 적재 스크립트

Claude Code로 적재 스크립트를 만들고, 메타데이터 누락이 0건이 될 때까지 고친다.

```
src/ingest.py를 만들어줘.
- data/raw/의 PDF·HWPX·DOCX를 파싱해 조항 단위로 분할
- 각 조항에 doc, article, effective_date, security_level, revision 메타데이터
- data/parsed/articles.jsonl로 저장
- 파싱 실패 문서와 메타데이터 누락 건수를 마지막에 출력
```

```
누락이 0건이 될 때까지 Claude Code와 함께 고친다
조항 경계가 맞는지는 3건 이상 직접 원문과 대조한다
```

실습 ② 평가셋 5건을 원문과 대조한다

평가셋의 정답 조항이 틀리면 이후 모든 점수가 틀린다. 그래서 일부라도 사람이 원문과 맞춰 본다.

JSON


```
{"id": "Q07", "type": "trap_revision", "question": "...",
"gold_answer": "...",
 "gold_citations": [{"doc": "시세정보전송규정", "article": "제12조",
"effective": "2026-03-01"}],
 "role": "general", "notes": "2025 개정 전 조항은 30초, 개정 후 15초"}
```


확인할 것

- gold_citations(정답 조항)의 문서, 조항, 시행일이 원문과 같은가
- gold_answer(정답)가 그 조항 내용과 맞는가

실습 ③ 평가 러너를 만든다

평가 러너는 그래프 모듈이나 결과 파일을 받아 같은 형식으로 기록한다. 이후 모든 방식을 이 러너 하나로 잰다.

src/run_eval.py를 만들어줘.
- 입력: LangGraph 그래프 모듈 경로 또는 결과 jsonl 경로
- 그래프면 평가셋 전체를 실행해 eval/results/<name>_r<n>.jsonl로 저장
- 기록 항목: 답변, 인용, 토큰, 지연
- 채점은 다음 세션에서 붙일 수 있도록 함수만 분리해 둬
- CLAUDE.md의 결과 형식을 따를 것

solutions의 샘플 그래프로 러너가 동작하는지 확인한다

확인 기준과 막힐 때

메타데이터 누락이 0건이고, 러너가 샘플 그래프로 결과 파일을 만들면 성공이다.

```
확인 기준
- 메타데이터 누락 0건
- 러너가 샘플 그래프로 결과 파일을 생성함

막힐 때
- HWPX 파싱 실패 → 정답의 파서를 사용
- 세션 종료 시   → git checkout s1-2-done 으로 맞추고 진행
```

적재 설계 | 평가 설계 | 적재 실습 | 평가 러너 실습

다음 시간: 하이브리드 검색

다음 시간에는 적재한 조항을 찾는 검색기를 만들고, 첫 번째 RAG 그래프를 만든다.

```
1-3 하이브리드 검색, 재순위 (11:00–12:00)
결과물: src/retriever.py, src/graphs/graph_rag.py
```

적재 설계 · 평가 설계 · 적재 실습 · 평가 러너 실습

SESSION 03 / 14

1-3

1일차
11:00-12:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

하이브리드 검색, 재순위

이번 시간에는 키워드 검색과 의미 검색을 합친 검색기를 만들고, 벡터 검색만 쓸 때보다 정답 조항을 더 많이 찾는지 비교한다.

학습 목표

1. **BM25와 벡터 검색의 강점과 약점을 직접 확인하고, 하이브리드 검색이 표준으로 쓰이는 이유를 설명할 수 있다**

결과물 · src/retriever.py, src/graphs/graph_rag.py

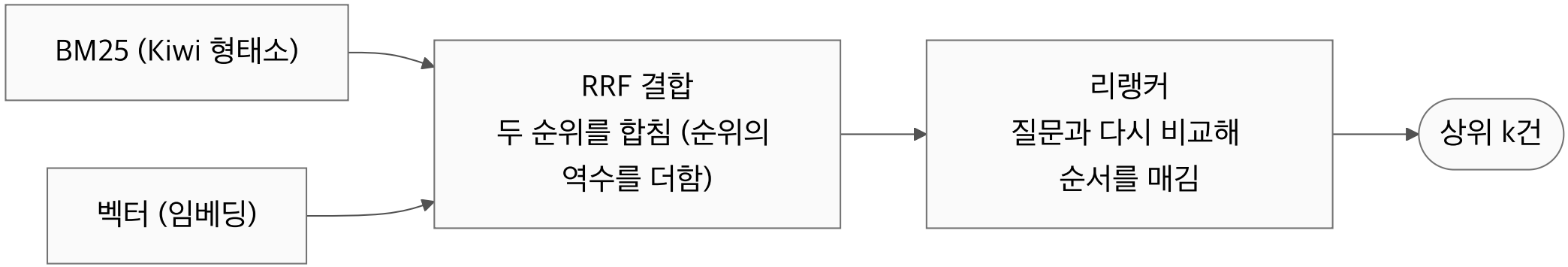
BM25와 벡터 검색의 차이

BM25는 조항 번호나 코드처럼 정확히 같은 단어를 잘 찾는다. 벡터 검색은 단어가 달라도 뜻이 비슷한 문장을 잘 찾는다.

	BM25 (키워드 검색)	벡터 검색 (의미 검색)
원리	같은 단어가 얼마나 나오는지로 점수 계산. 드문 단어일수록 가중치가 큼	문장을 숫자 벡터(임베딩)로 바꿔 뜻이 가까운 것을 찾음
잘 찾는 것	조항 번호, 시스템 코드, 시간 기준	표현이 다른 같은 뜻, 동의어
놓치기 쉬운 것	다른 단어로 쓴 같은 뜻	정확히 일치해야 하는 번호나 코드
필요 자원	CPU	임베딩 모델
평가셋에서 유리한 유형	단일 사실(고유한 표현)	함정: 표현 변형

두 검색 결과를 합치고 다시 정렬한다

업계에서는 두 검색 결과를 순위로 합친(RRF) 뒤 리랭커로 다시 정렬하는 구성을 많이 쓴다. 재무·감사 문서를 다룬 연구에서는 BM25가 벡터 검색보다 성적이 좋았다.



원문 요약

READ (재무·감사 문서): 정답률 READ 58.8%, dense 검색(벡터 검색) 15.7%

BM25와 READ는 통계적으로 차이가 없음

rag_research_report.md §2.5 [24] — https://arxiv.org/abs/2608.06305 · 원문 요약

검색 방식 비교 | 한국어 검색 | 검색기 실습 | RAG 그래프 실습

한국어 BM25에는 형태소 분석이 필요하다

한국어는 단어 뒤에 조사가 붙어서 공백으로만 나누면 같은 단어로 인식되지 않는다. 형태소 분석기로 나누면 BM25 점수가 오른다.

토크나이저 (문장을 단어로 나누는 도구)	top-1 F1
okt	0.798
kkma	0.754
kiwi	0.728
공백 분리	0.667

출처: rag_research_report.md §7.1 [106] · 원문 요약 (⚠ 2024년 데이터셋 1개 기준)

예: "참가사에게"와 "참가사는"은 공백으로 나누면 다른 단어가 된다
top-1 F1: 첫 번째 검색 결과가 정답인 정도를 나타내는 점수 (1에 가까울수록 좋음)
실습은 Kiwi를 쓴다. pip로 바로 설치할 수 있기 때문이다. 이 표는 2024년 데이터셋 1개 기준이다

한국어 임베딩 모델과 리랭커

작은 한국어 특화 임베딩 모델이 큰 다국어 모델보다 점수가 높았다. 한국어 리랭커도 금융 분야에서 원본 모델보다 나았다.

모델	크기	MTEB(kor, v2) 검색 nDCG@10
KURE-v2	154M	0.816
Qwen3-Embedding-8B	7.6B	0.783
bge-m3	568M	0.751

출처: rag_research_report.md §7.1 [103][104] · 원문 요약

MTEB: 임베딩 모델 공개 벤치마크. nDCG@10: 상위 10건의 순서가 얼마나 정확한지 나타내는 점수 (1에 가까울수록 좋음)
리랭커 bge-reranker-v2-m3-ko: 금융 분야 top-1 F1 0.912 (원본 모델 0.877)

[1-3] 하이브리드 검색, 재순위

실습: 하이브리드 검색과 RAG 그래프

실습 ① 검색 모듈을 만든다

네 가지 검색 방식과 메타데이터 필터를 함수 하나로 묶는다. 이후 모든 그래프가 이 검색기를 같이 쓴다.

```
src/retriever.py를 만들어줘.
- BM25 두 종류(공백 토큰, Kiwi 형태소), 벡터(로컬 임베딩 모델 + Chroma)
- RRF로 결합, 선택적으로 리랭커 적용
- hybrid_search(query, k=5, filters=None, mode="hybrid|bm25|bm25_kiwi|vector")
- filters는 security_level, effective_date 메타데이터 필터를 지원
```

검색 방식 비교 | 한국어 검색 | **검색기 실습** | RAG 그래프 실습

실습 ② 검색 방식별 적중 수를 비교한다

답을 만들지 않고 검색만 따로 재면, 어느 방식이 정답 조항을 찾아오는지 바로 비교할 수 있다.

```
평가셋 30건에 대해 mode별 상위 5건 안에 gold_citations 조항이
들어간 질문 수를 표로 출력하는 스크립트를 만들어줘.
리랭커 적용 전후도 포함해.
```

```
아래는 실습에서 직접 채우는 표다 (값: 상위 5건 안에 정답 조항이 들어간 질문 수)
```

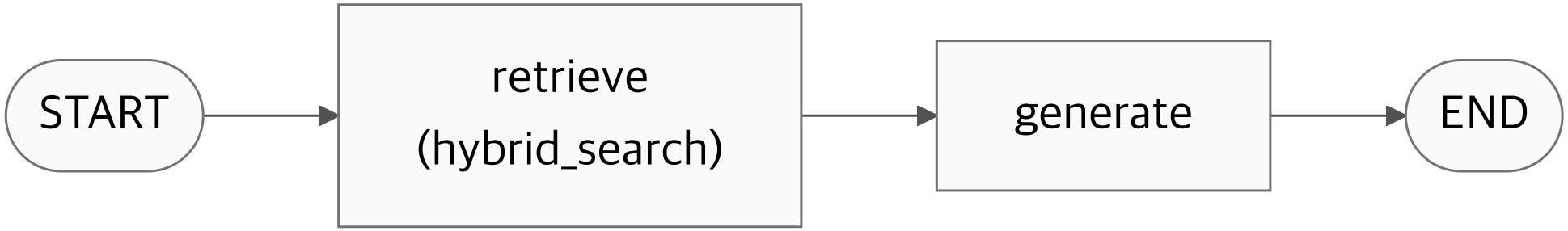
검색 방식	리랭커 없음	리랭커 적용
bm25 (공백)		
bm25_kiwi		
vector		
hybrid		

검색 방식 비교 · 한국어 검색 · **검색기 실습** · RAG 그래프 실습

실습 ③ 검색 후 답을 만드는 그래프

검색 노드와 답변 생성 노드 두 개로 된 그래프가 첫 번째 RAG 방식이다. 만든 뒤 평가 러너로 바로 측정한다.

```
src/graphs/graph_rag.py를 만들어줘. LangGraph StateGraph로
retrieve 노드(hybrid_search) → generate 노드. CLAUDE.md의 1.x API 예시를 따를 것.
```



검색 방식 비교 | 한국어 검색 | 검색기 실습 | **RAG 그래프 실습**

확인 기준과 막힐 때

하이브리드 검색의 상위 5건 적중 수가 벡터 검색만 쓸 때보다 많으면 성공이다.

```
확인 기준
- 하이브리드의 상위 5건 적중 수가 벡터 단독보다 많다
- 표현 변형 문항과 고유 토큰 문항에서 BM25와 벡터의
  승패가 갈리는 사례를 1개씩 찾는다

막힐 때
- 색인이 느림 → 미리 만든 Chroma 색인(solutions/index/) 사용
```

검색 방식 비교 | 한국어 검색 | 검색기 실습 | RAG 그래프 실습

다음 시간: 자동 채점과 채점 신뢰성

점심 뒤에는 기준선과 RAG의 결과를 RAGAS로 채점하고, 그 채점을 얼마나 믿을 수 있는지 확인한다.

```
2-1 자동 평가와 채점 신뢰성 (13:00–14:00)
결과물: 점수표 v1 (기준선 vs RAG)
```

검색 방식 비교 | 한국어 검색 | 검색기 실습 | RAG 그래프 실습

SESSION 04 / 14

2-1

1일차
13:00-14:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

자동 평가와 채점 신뢰성

이번 시간에는 기준선과 RAG를 같은 방법으로 채점해 점수표를 만들고, 그 점수가 실행마다 얼마나 달라지는지 확인한다.

학습 목표

1. **RAGAS로 개선 전후를 비교하고, 채점 결과를 그대로 믿지 않고 확인하는 방법을 익힌다**

결과물 · 점수표 v1 (기준선 vs RAG)

이 과정에서 쓰는 RAGAS 지표 두 가지

Faithfulness는 답이 검색된 문맥에 근거하는지를 본다. Context Recall은 정답에 필요한 내용이 검색되었는지를 본다.

지표	보는 것	방법	보지 않는 것
Faithfulness (근거 충실도)	답이 검색된 문맥에 근거하는가	답을 문장 단위 주장으로 나눠 각각 문맥으로 뒷받침되는지 확인	답이 사실로 맞는지
Context Recall (문맥 재현율)	정답에 필요한 내용이 검색되었는가	정답의 주장이 검색 결과에 있는지 확인	답변의 품질

RAGAS는 LLM이 채점하는 방식(LLM-as-judge)이다. 채점에도 LLM 호출 비용이 든다

Faithfulness가 1.0이어도 오답일 수 있다

검색이 개정 전 조항을 가져왔고 모델이 그대로 답했다면, 답은 틀렸지만 문맥과는 일치하므로 Faithfulness는 1.0이 나온다.

```
질문: 시세 전송 지연 시 참가사 통보 기준은?
검색: 시세정보전송규정 제12조 (개정 전) — "30초"
답변: 30초 이상 지연 시 통보한다
──────────────────────────────────────────────
Faithfulness = 1.0  (답이 검색 문맥과 일치)
실제 정답      = 15초 (개정 후 조항)  → 오답
```

채점 결과도 확인이 필요하다

LLM 채점은 실행할 때마다 결과가 달라지고, 채점 모델끼리도 판정이 다르다. 그래서 같은 채점을 반복해 편차를 보고, 사람이 채점한 결과와 맞춰 본다.

원문 요약

- ✓ CRAG [27]: 가장 좋은 시스템도 환각 없이 답한 비율이 63%
- ✓ Judge's Verdict [29]: LLM 채점을 검증할 때는 상관계수보다 Cohen's κ(두 채점이 실제로 같은 판정을 낸 정도) 같은 일치도를 봐야 함
- ✓ Reliability without Validity [30]: 판정이 일관적이라고(신뢰도) 해서 맞는 판정이라는(타당도) 보장은 없음
- ✓ Cochran [67]: v3에서 채점 버그를 고친 뒤 이전 결론을 철회

rag_research_report.md §2.6 [27][29][30], §4.5 [67] · 원문 요약

이번 실습: 같은 설정으로 2회 채점하고, 사람이 채점한 10건과 비교한다

[2-1] 자동 평가와 채점 신뢰성

실습: 채점과 신뢰성 확인

실습 ① 평가 러너에 RAGAS 채점을 붙인다

1-2에서 비워 둔 채점 함수에 RAGAS 지표 두 개를 붙인다. 기준선은 규정집 전체를 문맥으로 놓고 채점한다.

run_eval.py에 RAGAS faithfulness, context_recall 채점을 붙여줘.
- RAGAS 버전은 CLAUDE.md 고정 버전
- 결과 jsonl별 평균과 문항별 점수를 eval/results/scores.csv에 누적
- 기준선(Claude Code 결과)의 faithfulness 문맥은 data/regs_all.md 전체
- 기준선의 context_recall은 1.0으로 기록(전체 문서가 문맥이므로 정의상 1.0)

기준선의 context_recall이 1.0인 것은 규정집 전체를 넣었기 때문이다. 성능이 좋다는 뜻이 아니다

RAGAS 지표 | 채점 검증 | **채점 실습** | 신뢰성 확인 실습

실습 ② 편차, 사람 채점, 함정 문항 확인

같은 설정으로 두 번 채점한 편차, 사람 채점과 다른 사례, 함정 문항의 점수를 직접 보고 점수표 v1을 완성한다.

1. 기준선과 RAG를 각각 2회 채점해 편차를 확인한다
2. human_labels.csv 10건과 RAGAS 점수의 일치 여부를 표로 만든다
3. 함정 문항(개정 전 조항)의 점수를 확인한다

아래는 실습에서 직접 채우는 표다

방식	Faithfulness (평균 ± 편차)	Context Recall	함정 문항 오답	질문당 토큰
기준선		1.0		
하이브리드 RAG				

RAGAS 지표 · 채점 검증 · 채점 실습 · **신뢰성 확인 실습**

확인 기준과 막힐 때

점수표 v1이 나오고, 채점 모델과 사람이 다르게 판정한 사례 1건을 설명할 수 있으면 성공이다.

확인 기준
- 점수표 v1(기준선 vs RAG, 2회 평균 ± 편차)이 나온다
- judge와 사람이 불일치한 사례 1건을 설명할 수 있다

막힐 때
- RAGAS 버전 오류 → 정답의 채점 함수 사용
- 호출 한도 초과 → 10건 부분 집합으로 채점

RAGAS 지표 | 채점 검증 | 채점 실습 | 신뢰성 확인 실습

다음 시간: 출처 인용과 답변 거부

다음 시간에는 모든 답에 출처를 붙이게 하고, 근거가 부족하면 답하지 않게 만든다.

```
2-2 출처 인용, 답변 거부 (14:00–15:00)
결과물: src/answer_schema.py
```

RAGAS 지표 · 채점 검증 · 채점 실습 · 신뢰성 확인 실습

SESSION 05 / 14

2-2

1일차
14:00-15:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

출처 인용, 답변 거부

이번 시간에는 모든 답에 근거 조항을 붙이게 하고, 근거가 부족하면 답하지 않도록 RAG 그래프를 고친다.

학습 목표

1. **답에 인용을 정해진 형식으로 붙이게 하고, 근거가 없으면 답하지 않는 그래프를 만든다**

결과물 · src/answer_schema.py

금융권에서 출처 표시가 필요한 이유

금융분야 AI 가이드라인은 최종 판단을 직원이 하도록 정한다. 직원이 원문을 바로 확인할 수 있게 답에 문서명, 조항, 시행일을 붙인다.

원문 요약

금융분야 AI 가이드라인 (2026-06-22 시행, 자율규제)

7대 원칙 중 "보조수단성": 최종 결정과 책임은 직원에게 있음

rag_research_report.md §7.2 [107] — https://www.fsc.go.kr/no010101/87142 · 원문 요약

```
인용 형식 예시: [시세정보전송규정 제12조, 시행 2026-03-01]
```

출처 요건 | 스키마와 거부 엣지 | 인용·거부 실습

답, 인용, 근거 충분 여부를 한 형식으로

답을 자유 문장 대신 정해진 JSON 형식(구조화 출력)으로 받으면 인용이 빠졌는지를 코드로 검사할 수 있다. 이 형식은 이후 세션에서도 그대로 쓴다.

Python

```python
from pydantic import BaseModel

class Citation(BaseModel):
    doc: str              # 문서명
    article: str          # 조항
    effective_date: str   # 시행일

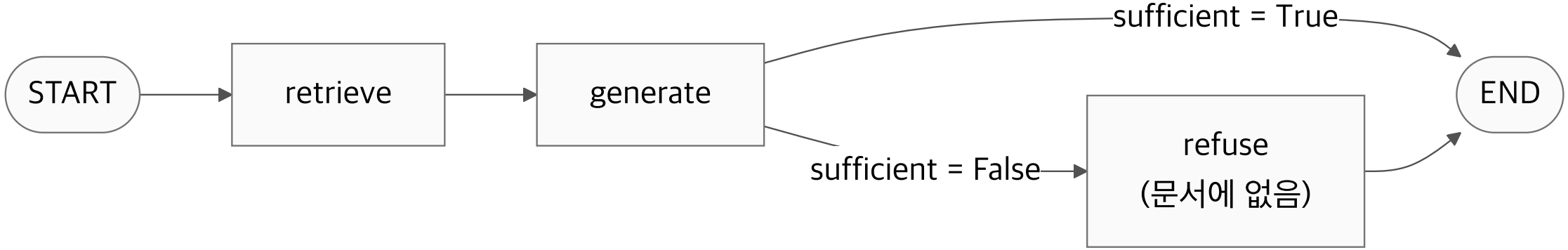
class Answer(BaseModel):
    answer: str
    citations: list[Citation]
    sufficient: bool      # 근거가 충분한가
```

구조화 출력	LLM이 정해진 JSON 형식으로 답하게 하는 기능
Pydantic 라이브러리	Python에서 데이터 형식을 정의하고 검사하는
재사용 verifier	2-3 graph_agentic, 과제 1 graph_wiki, 3-2

출처 요건 · 스키마와 거부 엣지 · 인용·거부 실습

근거가 부족하면 "문서에 없음"으로 보낸다

sufficient 값이 False이면 답을 만들지 않고 "문서에 없음" 노드로 보낸다. 이 분기는 조건부 엣지(State 값을 보고 다음 노드를 고르는 연결)로 만든다.



출처 요건 | 스키마와 거부 엣지 | 인용·거부 실습

[2-2] 출처 인용, 답변 거부

실습: 인용과 거부

실습 ① 스키마를 RAG 그래프에 적용하기

스키마와 거부 분기를 한 번에 지시한다. 만들어진 그래프에서 분기 조건이 맞는지 검수한다.

```
src/answer_schema.py에 Pydantic 스키마를 만들어줘:
answer, citations[{doc, article, effective_date}], sufficient(bool).
graph_rag.py의 generate를 구조화 출력으로 바꾸고,
sufficient=False면 조건부 엣지로 refuse 노드("문서에 없음")로 보내줘.
```

출처 요건 · 스키마와 거부 엣지 · **인용·거부 실습**

실습 ② 문서 밖 질문 10건과 평가셋 30건 실행

거부가 잘 되는지는 문서 밖 질문 10건으로 확인한다. 정상 질문의 품질이 유지되는지는 평가셋 30건을 다시 실행해 확인한다.

1. 문서 밖 질문 10건을 실행해 거부율을 확인한다
2. 평가셋 30건을 다시 실행해 근거 충실도 변화를 확인한다

	적용 전	적용 후
문서 밖 질문 거부율 (10건)		
Faithfulness (30건)		
인용 누락 건수		

실습에서 직접 채우는 표

출처 요건 · 스키마와 거부 엣지 · 인용·거부 실습

확인 기준과 막힐 때

거부율이 오르고 근거 충실도가 유지되거나 오르면 성공이다. 인용 누락은 0건이어야 한다.

```
확인 기준
- 문서 밖 질문 거부율이 오르고, 근거 충실도가 유지되거나 오른다
- 인용 누락 0건

막힐 때
- 모델에서 구조화 출력이 실패 → 정답의 프롬프트와 파서 사용
```

출처 요건 · 스키마와 거부 엣지 · 인용·거부 실습

다음 시간: 에이전틱 검색

다음 시간에는 에이전트가 검색 도구를 골라 여러 번 검색하는 방식을 만들고, 정확도와 비용을 측정한다.

```
2-3 에이전틱 검색 (15:00–16:00)
결과물: src/graphs/graph_agentic.py
```

출처 요건 | 스키마와 거부 엣지 | 인용·거부 실습

SESSION 06 / 14

2-3

1일차
15:00-16:00

강의 **15분**
체험 **15분**
구현 **20분**

RAG 기반 Multi-AI Agent 만들기

에이전틱 검색

이번 시간에는 에이전트가 검색 도구를 골라 여러 번 검색하는 방식을 체험하고 직접 만든다. 고정 RAG와 비교해 정확도와 비용이 얼마나 달라지는지 측정한다.

학습 목표

1. **에이전트가 검색 도구를 골라 반복하는 방식을 체험하고, 정확도와 비용을 측정한다**

결과물 · src/graphs/graph_agentic.py

에이전틱 검색이란

에이전틱 검색은 검색을 언제, 어떤 도구로, 몇 번 할지를 LLM이 결과를 보면서 정하는 방식이다. 1-3에서 만든 고정 RAG는 이 순서가 코드에 정해져 있다.

인용

Traditional approaches using RAG use static retrieval.

기존 RAG 방식은 정해진 순서대로 한 번 검색한다

rag_research_report.md §3.1 [33] — https://www.anthropic.com/engineering/multi-agent-research-system

	고정 RAG (1-3)	에이전틱 검색
검색 횟수	질문당 1회	결과를 보고 반복
검색 도구	코드에 고정	에이전트가 선택 (grep, 파일 읽기, 하이브리드 검색)
검색어	질문 그대로	에이전트가 다시 씀
비용과 지연	작고 일정함	크고 질문마다 다름

에이전틱 검색의 네 가지 형태

에이전틱 검색은 네 가지 형태로 나눌 수 있다. 이번 실습에서는 1번을 체험하고 2번을 구현한다.

형태	예시	근거	이번 과정
1. 인덱스 없이 원문을 직접 탐색	grep(파일 안 문자열 검색), 파일 읽기	Anthropic [41], Cline [44], DCI [22]	2-3 체험
2. 여러 검색 도구 중 선택	grep + 임베딩 검색	LlamaIndex [46], Cursor [48]	2-3 구현
3. 검색 전담 서브에이전트	읽기만 하는 검색 담당 에이전트	Anthropic [33], Cognition [49], Chroma [60]	3-2
4. 강화학습(RL)으로 학습한 검색 정책	Search-R1 등	Search-R1 [18], SearchArt [21]	과제 2 소개

Claude Code의 검색 방식

Claude Code는 인덱스 없이 CLAUDE.md만 미리 읽고, 나머지는 필요할 때 glob과 grep으로 찾는다. 체험 실습에서 이 탐색 경로를 직접 관찰한다.

원문 요약

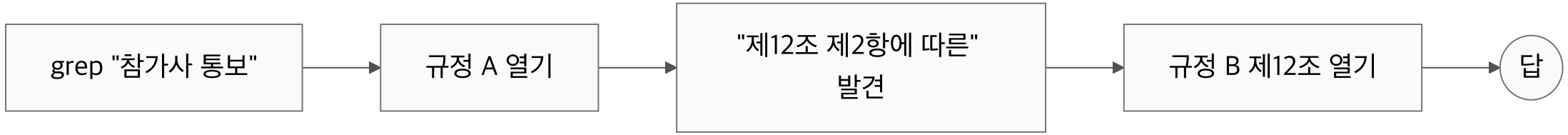
Claude Code는 CLAUDE.md를 미리 로드하고, 나머지는 glob(파일 이름 패턴 검색)과 grep(파일 내용 검색)으로 찾음

✓ [34]: 미리 임베딩해 두는 대신 필요할 때 도구로 불러오는 just-in-time 검색.

✓ [34]: 법률·금융처럼 내용이 덜 바뀌는 영역에는 하이브리드가 적합하다고 명시

✓ [41]: Claude Code는 인덱스가 필요 없음. 한계: 처음에 줄 맥락(CLAUDE.md)이 필요

rag_research_report.md §3.1 [34][41] · 원문 요약



에이전틱 검색이란 | 근거와 비용 | 직접 탐색 체험 | 도구 루프 구현

정확도와 비용에 관한 연구 결과

grep으로 직접 탐색하는 방식이 기존 검색보다 나았다는 연구가 있다. 대신 질의당 시간과 입력 토큰이 크게 늘었다.

원문 요약

검색 벤치마크 BRIGHT와 BEIR에서 키워드 검색, 벡터 검색, 리랭크 기준선보다 높은 점수
(일반 검색 0.67초), 입력 약 76만 토큰

- ✓ DCI [22]: 인덱스 없이 grep, 셸, 파일 읽기로 문서를 직접 탐색.
- ✓ Cursor [48] (자체 측정): grep에 임베딩 검색을 더하면 정확도 +12.5% (모델에 따라 6.5~23.5%)
- ✓ 에이전틱 검색의 비용 [25]: 검색 품질 점수(nDCG@10) +8.7, 대신 질의당 약 107초

rag_research_report.md §2.5 [22][25], §3.2 [48] · 원문 요약

금융 관점에서 본 장점과 약점

에이전트가 어떤 파일을 왜 열었는지 로그로 남으므로 감사 추적에는 유리하다. 하지만 실행할 때마다 경로가 달라서 같은 답을 재현하기 어렵다.

	내용	관련 조건
장점	사용한 도구, 인자, 열어 본 파일이 로그로 남아 감사 추적에 쓸 수 있음	출처, 측정
약점	실행마다 경로가 달라 재현성이 떨어짐	측정
주의	grep과 파일 읽기는 벡터 DB의 권한 필터를 거치지 않음. 4-2에서 경로 제한으로 막음	권한

[2-3] 에이전틱 검색

실습: 체험과 구현

실습 ① 원문 폴더를 직접 탐색하기

색인 없이 원문 폴더만 주고 질문한다. 에이전트가 grep으로 찾고, 파일을 열고, 교차 참조를 따라가는 경로를 관찰한다.

```
data/raw/ 폴더만 사용해서 답해줘. 색인이나
data/parsed는 쓰지 마.
질문: (교차 참조 문항 1개)
어떤 파일을 왜 열었는지 단계별로 보여줘.
```

```
관찰 기록: grep 검색어 → 연 파일 → 따라간 교차 참조 →
최종 인용
```

실습 ② (선택) 직접 탐색 방식 일괄 실행

시간이 남을 때만 직접 탐색 방식으로 평가셋을 실행해 참고 점수를 얻는다. 이 점수는 4방식 점수표에 넣지 않는다.

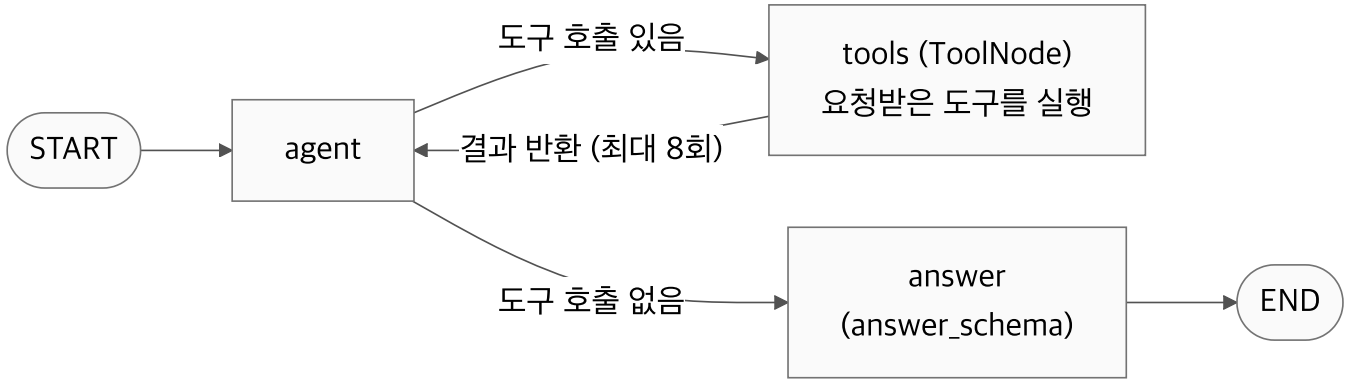
```
scripts/run_claude_explore.sh를
run_claude_baseline.sh처럼 만들어줘.
허용 도구는 Read, Grep, Glob. 대상은 data/raw/.
결과는 explore_r1.jsonl.
```

선택 실습. 시간이 부족하면 건너뛴다

실습 ③ 도구를 반복 호출하는 그래프 만들기

에이전트가 세 도구 중 하나를 골라 호출하고, 결과를 보고 다시 고르는 그래프를 만든다. 비용이 끝없이 늘지 않도록 최대 반복 횟수를 정한다.

```
src/graphs/graph_agentic.py를 만들어줘.
- 도구: grep(pattern, path), read_file(path), hybrid_search(query, k, filters)
- LangGraph ToolNode + 조건부 엣지 루프, 최대 반복 8회
- 최종 답은 answer_schema 구조화 출력
- 도구 호출 로그(도구명, 인자, 결과 길이)를 state에 누적
```



확인 기준과 막힐 때

고정 RAG가 틀린 질문 중 몇 개를 맞혔는지 센다. 질문당 토큰과 지연이 몇 배 늘었는지도 수치로 답하면 성공이다.

확인 기준
- 고정 RAG가 틀리고 에이전틱 검색이 맞힌 질문 수를 센다
- 질문당 토큰과 지연이 몇 배 늘었는지 수치로 답한다

막힐 때
- 반복이 끝나지 않음 → 최대 반복 수와 종료 조건을 정답 코드로 교체
- 시간 부족 → 실습 ②(헤드리스)는 생략

에이전틱 검색이란 | 근거와 비용 | 직접 탐색 체험 | 도구 루프 구현

다음 시간: llm-wiki와 4방식 비교

다음 시간에는 규정을 미리 위키 페이지로 정리해 두고 위키를 읽고 답하는 방식을 만든다. 그리고 1일차의 4방식을 같은 평가셋으로 비교한다.

```
과제 1 llm-wiki, 4방식 비교 (16:00–17:00)
결과물: wiki/ · src/graphs/graph_wiki.py · 4방식 점수표
```

에이전틱 검색이란 | 근거와 비용 | 직접 탐색 체험 | 도구 루프 구현

SESSION 07 / 14

과제 1

1일차
16:00-17:00

강의 **15분**

실습 **35분**

RAG 기반 Multi-AI Agent 만들기

llm-wiki, 4방식 비교

이번 시간에는 규정을 위키로 정리하는 llm-wiki를 만들어 본다. 그리고 1일차에 만든 4방식을 질문 유형별 점수로 비교한다.

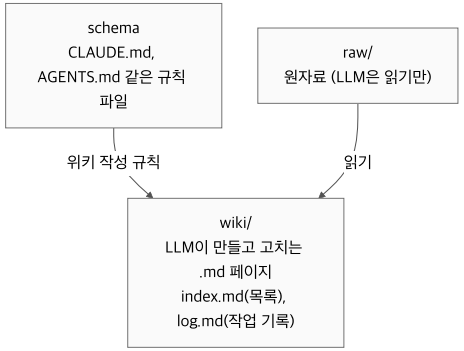
학습 목표

1. **llm-wiki를 직접 만들고, 금융권에 쓰려면 무엇을 보완해야 하는지 판단한다**
2. **기준선, RAG, 에이전틱 검색, 위키의 4방식을 같은 평가셋으로 비교한다**

결과물 · wiki/ · src/graphs/graph_wiki.py · 4방식 점수표

llm-wiki의 구조

llm-wiki는 원자료, 위키, 규칙 파일의 세 층으로 이루어진다. LLM은 원자료를 읽기만 하고, 규칙 파일에 따라 위키 페이지를 만들고 고친다.



작업	내용
ingest (반영)	원자료 1건을 읽고 위키 10~15페이지를 갱신
query (질의)	위키로 답하고, 좋은 답변은 위키에 다시 넣음
lint (점검)	모순, 낡은 내용, 다른 페이지와 연결되지 않은 페이지를 점검

출처: rag_research_report.md §4.2 [66] · 원문 요약

Karpathy가 설명한 RAG와 위키의 차이

Karpathy는 RAG가 질문할 때마다 지식을 처음부터 다시 찾는다고 보았다. 위키는 한 번 정리해 두고 계속 갱신한다는 점에서 다르다고 설명했다.

인용

the LLM is rediscovering knowledge from scratch on every question. There's no accumulation.

LLM은 질문할 때마다 지식을 처음부터 다시 찾는다. 쌓이는 것이 없다.

rag_research_report.md §4.2 [66] — https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

인용

The knowledge is compiled once and then *kept current*, not re-derived on every query.

지식은 한 번 정리된 뒤 최신 상태로 유지된다. 질의할 때마다 다시 만들지 않는다.

rag_research_report.md §4.2 [66] — https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

llm-wiki 구조 | 실증과 비판 | 금융권 적용 원칙 | 위키 컴파일 | 위키 에이전트 | 4방식 비교

원저자가 밝힌 적용 범위

Karpathy는 약 100개 소스 규모에서 잘 작동한다고 했다. 위키가 커지면 BM25나 벡터 검색을 붙이라고 했으므로, RAG를 대체하자는 주장은 아니다.

인용

~100 sources, ~hundreds of pages

소스 약 100개, 페이지 수백 개 규모

rag_research_report.md §4.2 [66] — https://gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

원문 요약

위키가 커지면 qmd(BM25, 벡터, LLM 리랭크를 묶은 검색 도구)를 붙이라고 함

rag_research_report.md §4.2 [66] · 원문 요약

확산 현황과 비슷한 사례

원 트윗 직후 GitHub 구현이 많이 나왔고, 벡터 DB와 프레임워크 업체도 비슷한 제품을 냈다. 코드 위키와 Claude Code memory도 같은 형태다.

확산	비슷한 사례
nashsu/llm_wiki ★20,685 [75]	DeepWiki (Cognition): 코드 저장소 위키 자동 생성 [76]
nvk/llm-wiki ★1,417: 수집은 병렬, 정리는 순차 [74]	Google Code Wiki (2025-11-13) [77]
LangChain Wiki Memory (2026-06-30) [58]	Claude Code memory: MEMORY.md 목록 + 주제별 파일 [78]
Pinecone Nexus GA (2026-08), 수치는 자체 측정 [62]	Agent Skills: 필요할 때만 본문을 읽는 구조 [36]

llm-wiki 구조 | 실증과 비판 | 금융권 적용 원칙 | 위키 컴파일 | 위키 에이전트 | 4방식 비교

위키와 RAG를 비교한 연구

통제된 조건에서 위키와 RAG를 비교한 연구는 1건이고, 어느 쪽이 항상 낫다는 결론은 나오지 않았다. 종합 질문은 위키가, 단일 사실은 RAG가 나았고, 위키는 토큰을 약 21배 썼다.

원문 요약

- ✓ 여러 논문을 종합하는 질문은 wiki가 우세. 단일 사실 질문의 근거성은 RAG가 우세했으나 채점 LLM(judge)에 따라 달라짐
- ✓ wiki는 질의당 토큰을 약 21배 씀. 비용을 회수하는 지점 없음
- ✓ 질문을 쪼개 여러 번 검색하는 RAG 변형이 종합 성능 격차를 대부분 좁힘
- ✓ 한계: 저자 1인, 논문 24편, 질문 13개. v3에서 채점 버그 수정 후 이전 결론 철회

rag_research_report.md §4.5 [67] — https://arxiv.org/abs/2605.18490 · 원문 요약

```
소규모 연구: 논문 24편, 질문 13개
```

위키 방식의 약점

위키를 만드는 단계(ingest)에서 생긴 오류는 여러 페이지로 퍼질 수 있다. 또 사용자 역할별 권한이 없고, 원문 출처를 추적하기 어렵다.

인용

LLM Wiki can hallucinate at ingest time, which is more subtle and more dangerous.

LLM Wiki는 위키를 만드는 단계에서 환각을 일으킬 수 있고, 이는 더 알아차리기 어렵고 더 위험하다.

rag_research_report.md §4.5 [68] — https://dev.to/rosgluk/llm-wiki-compiled-knowledge-that-rag-cannot-replace-8op

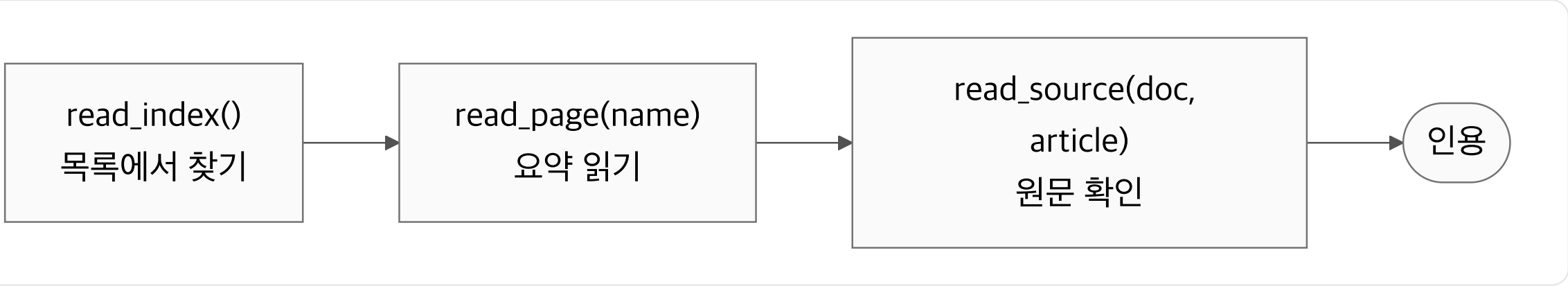
원문 요약

기업 적용 한계 (벤더 글): 목록 파일이 컨텍스트를 넘침, 역할 기반 권한이 없음, 규모가 커지면 내용이 낡음

rag_research_report.md §4.5 [69] — https://atlan.com/know/llm-wiki-vs-rag-knowledge-base/ · 원문 요약

금융권에서 위키를 쓰는 방법

위키는 어느 조항을 볼지 찾는 데 쓴다. 최종 인용은 원문 조항을 다시 읽어 확인한 것만 허용한다.



SCHEMA 규칙	막는 약점
모든 문장에 원문 근거 [doc §article, 시행일]를 붙임	ingest 단계 오류, 출처 추적
개정 전 조항은 "이력" 절로 분리	현행성
보안 등급이 다른 내용은 한 페이지에 섞지 않음	권한

[과제 1] llm-wiki, 4방식 비교

실습: 위키 컴파일과 4방식 비교

실습 ① 규칙 파일 작성과 문서 반영

금융 규칙을 담은 규칙 파일(SCHEMA.md)을 먼저 쓴다. 문서 3~5개만 직접 반영(ingest)하고, 나머지는 미리 만든 위키로 채운다.

```
wiki/SCHEMA.md를 만들어줘. Karpathy llm-wiki
패턴을 따르되 다음을 규칙으로:
- wiki/pages/의 모든 문장 끝에 원문 근거 [doc
§article, 시행일] 링크 필수
- 개정된 조항은 현행만 본문에, 개정 전은 "이력" 절로
- 보안 등급이 다른 내용은 한 페이지에 섞지 않음
- index.md(전체 목록), log.md(append-only) 유지
```

```
wiki/SCHEMA.md 규칙에 따라 data/raw/의 (지정
문서)를 ingest해서 wiki를 갱신해줘.
```

실습 ② 위키 점검하기 (lint)

lint로 근거 없는 문장, 페이지 간 모순, 낡은 내용, 고아 페이지(다른 페이지와 연결되지 않은 페이지)를 찾는다. ingest 단계의 오류는 이 점검으로 잡는다.

```
wiki를 lint해줘: 근거 링크 없는 문장, 페이지 간 모순, 시행일 지난 내용, 고아 페이지.
```

```
lint 결과: 근거 없는 문장 __건 / 모순 __건 / 낡은 내용 __건 / 고아 페이지 __건
```

실습 ③ 위키를 읽는 에이전트 만들기

위키로 볼 조항을 찾되, 최종 인용은 read_source로 원문을 확인한 조항만 허용하는 그래프를 만든다.

```
src/graphs/graph_wiki.py를 만들어줘.
- 도구: read_index(), read_page(name), read_source(doc, article)
- 위키로 탐색하되 최종 citations는 read_source로 확인한 원문 조항만 허용
- answer_schema 구조화 출력
```

실습 ④ 4방식 점수표 만들기

같은 평가셋 30건의 결과를 질문 유형별로 나눠 비교한다. 방식마다 점수가 높은 질문 유형이 다르게 나온다.

방식	단일 사실	교차 참조	종합	함정	질문당 토큰	지연
기준선						
하이브리드 RAG						
에이전틱 검색						
llm-wiki						

실습에서 직접 채우는 표

확인 기준과 막힐 때

질문 유형별로 어느 방식의 점수가 높은지 답하면 성공이다.

```
확인 기준
- 질문 유형별(단일 사실, 교차 참조, 종합, 함정)로 어느 방식이
  우세한지 점수로 답한다

막힐 때
- ingest 시간 초과 → solutions/wiki/ 전체 사용
- 시간 부족        → 4방식 점수표는 2일차 09시 전에 강사가 정답 결과로 공유
```

llm-wiki 구조 · 실증과 비판 · 금융권 적용 원칙 · 위키 컴파일 · 위키 에이전트 · 4방식 비교

내일: 외부 지식 도구와 멀티 에이전트

2일차에는 오늘 만든 그래프와 부품을 멀티 에이전트의 서브에이전트와 도구로 다시 쓴다.

```
2일차 3-1 외부 지식 도구: CLI 스크립트 vs MCP (09:00–10:00)
결과물: day2/dart_cli.py · day2/dart_mcp.py
```

llm-wiki 구조 | 실증과 비판 | 금융권 적용 원칙 | 위키 컴파일 | 위키 에이전트 | 4방식 비교

SESSION 08 / 14

3-1

2일차
09:00-10:00

1일차 회고 **5분**
강의 **10분**
실습 **35분**

RAG 기반 Multi-AI Agent 만들기

외부 지식 도구: CLI 스크립트 vs MCP

이번 시간에는 DART 공시 조회 기능을 CLI 스크립트와 MCP 서버 두 가지로 만들고, 같은 질문으로 두 방식을 비교한다.

학습 목표

1. **같은 기능을 CLI 스크립트와 MCP 서버로 각각 연결한다**
2. **도구 정의에 드는 토큰, 호출 정확도, 재사용성, 권한 범위를 기준으로 두 방식을 비교한다**

결과물 · dart_cli.py · dart_mcp.py · 비교표

1일차 회고: 4방식 점수표

1일차 4방식 점수표에서는 질문 유형마다 점수가 가장 높은 방식이 달랐다. 2일차에는 여기에 외부 지식과 운영 조건을 더한다.

방식	단일 사실	교차 참조	종합	함정(개정)	질문당 토큰	지연
통째 넣기						
하이브리드 RAG						
에이전틱 검색						
llm-wiki						

강사가 1일차 실제 결과로 채움 (⚠)

1일차 회고 | 두 연결 방식 | 금융 관점 | CLI·MCP 비교 실습

[3-1] 외부 지식 도구: CLI 스크립트 vs MCP

두 가지 연결 방식

CLI 스크립트와 MCP 서버의 차이

CLI 방식에서는 에이전트가 셸에서 스크립트를 실행하고 출력 텍스트를 받는다. MCP 방식에서는 정해진 규약을 따르는 서버를 만들고, 여러 클라이언트가 그 서버를 그대로 연결해 쓴다.

CLI 스크립트

에이전트 → 셸 도구로 실행 → dart_cli.py

← stdout (JSON)

사용법은 시스템 프롬프트에 적는다

MCP 서버

에이전트 → MCP 클라이언트 → dart_mcp.py

← 도구 결과

도구 이름, 설명, 입력 형식을 서버가 알려 준다.
같은 서버를 Claude Desktop 등에도 연결할 수 있다

CLI	명령줄(셸)에서 실행하는 프로그램
MCP	에이전트와 도구를 연결하는 표준 규약 (Model Context Protocol)
stdio	표준 입출력으로 주고받는 통신 방식. MCP 서버도 이 방식을 쓸 수 있다
M×N → M+N	클라이언트 M개와 도구 N개를 각각 연동하지 않고, 표준 하나에 맞추면 M+N개만 만들면 된다

MCP는 도구 정의만큼 토큰을 더 쓴다

MCP 방식은 연결된 도구의 정의가 모두 컨텍스트에 들어가므로 토큰을 더 많이 쓴다. Anthropic은 이를 줄이는 방법으로 코드 실행과 Tool Search를 발표했다.

원문 요약

- ✓ Anthropic, Code execution with MCP (2025-11-04) [38]: 예시 작업에서 토큰 98.7% 절감
- ✓ Anthropic, Tool Search (2025-11-24) [39]: 컨텍스트 85% 절감. Opus 4.5의 MCP 평가 79.5% → 88.1%

rag_research_report.md §3.1 [38][39] · 원문 요약

원문 요약

MCP 방식은 도구 정의가 컨텍스트에 모두 들어가 CLI 방식보다 토큰을 4~32배 사용

Scalekit 블로그 — 레포트 외 출처, 벤더 자체 측정, 원문 미대조 ⚠
— https://www.scalekit.com/blog/mcp-vs-cli-use · 원문 요약

금융 환경에서 두 방식을 고르는 기준

사내 시스템을 여러 에이전트가 함께 쓴다면 MCP 서버가 권한 관리와 감사 기록에 유리하다. 커뮤니티가 만든 MCP 서버는 출처, 권한, 버전 고정을 확인하기 전에는 쓰지 않는다.

상황	권장	이유
사내 시스템을 여러 에이전트와 클라이언트가 함께 씀	MCP 서버	표준 인증, 감사 로그, 재사용
개인 작업용, 빠른 시험	CLI 스크립트	도구 정의 토큰이 적고 만들기 쉬움
커뮤니티 MCP 서버 (예: DART 비공식 구현)	확인 전에는 쓰지 않음	출처, 권한, 버전 고정을 확인해야 함

[3-1] 외부 지식 도구: CLI 스크립트 vs MCP

실습: CLI 스크립트 vs MCP 서버

실습 ① 같은 기능을 두 방식으로 만들기

두 방식의 기능은 같게 하고 연결 방식만 다르게 만들어야 비교할 수 있다.

```
day2/dart_cli.py를 만들어줘. argparse 서브커맨드 search(corp, days), document(rcept_no).
OpenDART API 호출, 결과는 JSON으로 stdout 출력. 키는 환경 변수.
```

```
같은 기능을 day2/dart_mcp.py로 FastMCP 서버(stdio)로 만들어줘.
도구 설명문과 입력 스키마를 명확히 작성.
```

```
FastMCP: Python 함수에 데코레이터를 붙여 MCP 서버를 만드는 라이브러리
```

실습 ② 두 에이전트를 질문 10건으로 비교

같은 질문 10건으로 두 에이전트를 실행하고, 도구 정의 토큰, 맞는 호출 비율, 인자 오류 수, 지연 시간을 잰다.

```
LangGraph 에이전트 2개를 만들어줘.
A: 셸 실행 도구 하나로 dart_cli.py를 호출 (사용법은 시스템 프롬프트에)
B: langchain-mcp-adapters로 dart_mcp.py 연결
DART 단독 질문 10건을 둘 다 실행하고
도구 정의 토큰, 맞는 호출 비율, 인자 오류 수, 지연을 표로 출력해줘.
```

방식	도구 정의 토큰	맞는 호출 비율	인자 오류 수	지연	재사용성	권한 범위
A. CLI 스크립트						
B. MCP 서버						

실습에서 직접 채우는 표

확인 기준과 막힐 때

답에 공시 번호와 제출일이 붙고, 두 방식의 차이를 수치로 설명할 수 있으면 성공이다.

확인 기준
- 답에 공시 번호와 제출일이 붙는다
- 두 방식의 차이를 수치로 답한다
- 재사용성과 권한 범위의 차이를 설명한다

막힐 때
- DART 키나 네트워크 문제 → `solutions/dart_cache/`의 녹화된 응답(mock)으로 실행

다음 시간: 규정 담당과 공시 담당 묶기

다음 시간에는 1일차 규정 에이전트와 이번에 만든 공시 도구를 감독자와 검증자로 묶고, 단일 에이전트와 비교한다.

```
3-2 감독자 구성, 단일 에이전트와 비교 (10:00–11:00)
결과물: day2/agents.py
```

1일차 회고 · 두 연결 방식 · 금융 관점 · CLI·MCP 비교 실습

SESSION 09 / 14

3-2

2일차
10:00-11:00

강의 **15분**

실습 **35분**

RAG 기반 Multi-AI Agent 만들기

감독자 구성, 단일 에이전트와 비교

이번 시간에는 규정 담당과 공시 담당 에이전트를 감독자와 검증자로 묶고, 같은 질문으로 단일 에이전트와 비교한다.

학습 목표

1. **멀티 에이전트가 단일 에이전트보다 나은 조건을 수치로 판단한다**
2. **인용, 시행일, 권한을 점검하는 검증자를 설계한다**

결과물 · day2/agents.py · 단일 vs 멀티 비교표

[3-2] 감독자 구성, 단일 에이전트와 비교

멀티 에이전트를 쓸 것인가

찬성 측: Anthropic의 측정 결과

Anthropic은 멀티 에이전트 리서치 시스템이 단일 에이전트보다 90.2% 높은 성능을 냈다고 보고했다. 대신 토큰은 일반 채팅의 약 15배를 썼다.

원문 요약

- ✓ 단일 에이전트 대비 +90.2%
- ✓ 토큰은 일반 채팅의 약 15배
- ✓ 의존성이 많은 작업이나 코딩에는 맞지 않음

rag_research_report.md §6.1 [33] — Anthropic, How we built our multi-agent research system (2025-06) · 원문 요약

멀티 에이전트	여러 에이전트가 역할을 나눠 일하는 구조
감독자	질문을 받아 어느 에이전트에게 맡길지 정하고 결과를 모으는 에이전트 (supervisor)
서브에이전트	감독자에게 일을 받아 처리하고 결과를 돌려주는 에이전트

찬반 근거 | 실증과 실패 | 설계 원칙 | 감독자 구성 실습

반대 측: Cognition의 주장

Cognition은 에이전트를 나누면 각자 내린 판단이 서로 어긋날 수 있다고 보았다. 그래서 에이전트끼리 전체 실행 기록을 공유해야 한다고 주장했다.

인용

Don't Build Multi-Agents

멀티 에이전트를 만들지 말라

rag_research_report.md [92] — https://cognition.com/blog/dont-build-multi-agents

인용

Share full agent traces

에이전트의 전체 실행 기록을 공유하라

rag_research_report.md [92] — https://cognition.com/blog/dont-build-multi-agents

인용

Actions carry implicit decisions

각 행동에는 드러나지 않은 판단이 담겨 있다

rag_research_report.md [92] — https://cognition.com/blog/dont-build-multi-agents

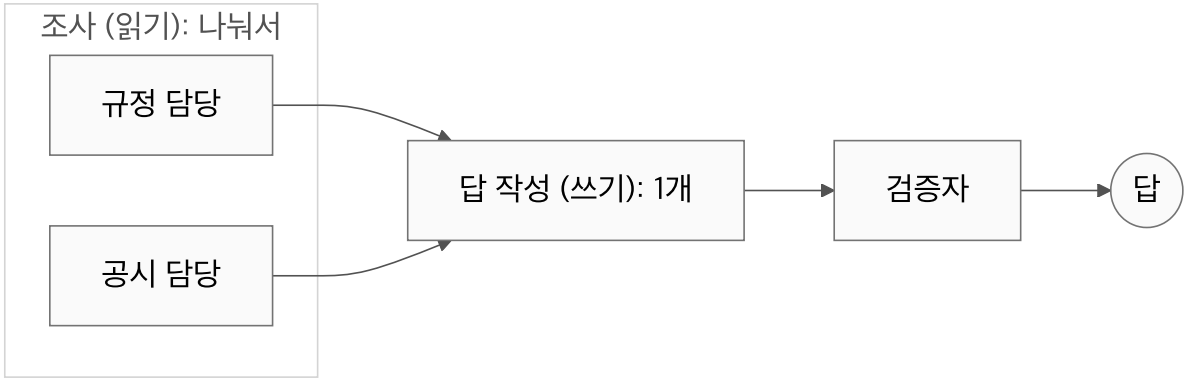
조사는 나눠서, 답 작성은 한 곳에서

두 입장이 함께 받아들이는 구성은 자료 조사를 여러 에이전트가 나눠 하고, 최종 답 작성은 한 에이전트가 맡는 방식이다.

원문 요약

- ✓ LangChain (2025-06) [93]: 읽기 작업은 병렬로 나누기 쉽고, 쓰기 작업은 나누기 어려움
- ✓ Cognition (2026-04) [94]: 가장 흔한 성공 사례는 읽기만 하는 검색 서브에이전트

rag_research_report.md §6.1 [93][94] · 원문 요약



찬반 근거 | 실증과 실패 | 설계 원칙 | 감독자 구성 실습

나눌 수 있는 과제에서만 성능이 올랐다

여러 부분으로 나눌 수 있는 금융 추론 과제에서는 성능이 크게 올랐다. 순서대로 진행해야 하는 계획 과제에서는 오히려 크게 떨어졌다.

원문 요약

✓ Kim 외 스케일링 연구 (260개 구성) [96]: 나눌 수 있는 금융 추론 +80.8%, 순차 계획 −70.0%

✓ Fudan 연구 [97]: 에이전트 수를 늘리면 성능이 오르다가 다시 떨어짐(역U자형)

rag_research_report.md §6.1 [96][97] · 원문 요약

역U자형: 처음에는 오르다가 어느 지점을 넘으면 다시 내려가는 모양

멀티 에이전트 실패 원인 분석

MAST 연구에서 멀티 에이전트 실패의 주된 원인은 모델 성능보다 역할 설계와 에이전트 간 조율 문제였다.

원문 요약

✓ MAST (2025-03) [95]: 실패 유형 14개를 3범주로 분류. 주원인은 LLM 한계가 아니라 설계와 조율

✓ LangChain 벤치마크 (2025-06) [102]: 감독자 구조는 답을 다시 전달하는 과정에서 손실이 생김

rag_research_report.md §6.1 [95][102] · 원문 요약

이번 실습의 대응

- 서브에이전트는 요약과 근거(evidence)만 반환한다
- 검증자가 인용을 원문으로 다시 확인한다
- 확인에 실패하면 해당 서브에이전트로 한 번만 되돌린다

서브에이전트는 요약만 돌려준다

서브에이전트가 탐색 과정 전체를 넘기면 감독자의 컨텍스트가 커진다. 그래서 1~2천 토큰 분량의 요약과 근거만 반환한다.

원문 요약

서브에이전트는 1~2천 토큰 요약만 돌려줌. 산출물 자체는 외부 저장소에 두고 참조만 전달

rag_research_report.md §6.2 [33][34] — Anthropic (2025-06, 2025-09) · 원문 요약

아키텍처 (LangChain 멀티 도메인 예시 [98])	토큰 사용량
Subagents	약 9K
Skills	약 15K
Handoffs	약 14K+

검증자의 역할

검증자는 최종 답의 인용을 원문으로 다시 읽고 시행일과 보안 등급을 점검한다. 금융 4조건 중 출처, 권한, 현행성을 이 단계에서 확인한다.

검증 항목	방법	확인하는 조건
인용 실재	인용한 조항을 read_source로 다시 읽음	출처
시행일	인용 조항이 현재 시행 중인지 확인	현행성
보안 등급	사용자 역할로 볼 수 있는 문서인지 확인	권한

검증자(verifier): 다른 에이전트가 만든 답을 원문과 대조해 점검하는 에이전트

[3-2] 감독자 구성, 단일 에이전트와 비교

실습: 감독자 구성

실습 ① 감독자, 서브에이전트, 검증자 구성

1일차 그래프와 3-1에서 만든 도구를 서브에이전트로 다시 써서 감독자 구조를 만든다.

```
day2/agents.py를 만들어줘. LangGraph supervisor 구조:
- regulation_agent: 1일차 graph_agentic(또는 graph_wiki)을 서브그래프로
- disclosure_agent: 3-1의 MCP 도구 사용
- verifier: 최종 답의 citations를 read_source로 재확인, 시행일·보안등급 점검,
  실패 시 해당 서브에이전트로 되돌림(최대 1회)
- 서브에이전트는 요약과 evidence만 반환
```

실습 ② 단일 에이전트와 같은 10건으로 비교

모든 도구를 가진 단일 에이전트를 만들고, 결합 질문 10건으로 정확도, 지연 시간, 토큰을 비교한다.

1. 비교용 단일 에이전트를 만든다 (모든 도구를 가진 에이전트 1개)
2. 결합 질문 10건을 둘 다 실행하고 정확도, 지연, 토큰을 표로 만든다

구조	정확도	질문당 토큰	지연	검증자가 잡은 건수
단일 에이전트				-
감독자 + 서브에이전트				

실습에서 직접 채우는 표

확인 기준과 막힐 때

어느 구조를 쓸지 수치로 설명하고, 검증자가 잡아낸 사례를 하나 설명할 수 있으면 성공이다.

확인 기준
- 어느 구조를 쓸지 수치로 답한다
- 검증자가 잡아낸 사례 1건을 설명한다

막힐 때
- 서브그래프 연결 오류 → 정답의 `agents.py`로 교체

다음 시간: 내부와 외부 근거를 한 답에 담기

다음 시간에는 근거를 공유하는 state를 만들어 중복 검색과 출처 누락을 줄인다.

```
3-3 내부·외부 지식 결합 (11:00–12:00)
결과물: 결합 규칙 · evidence state
```

찬반 근거 | 실증과 실패 | 설계 원칙 | 감독자 구성 실습

SESSION 10 / 14

3-3

2일차
11:00-12:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

내부·외부 지식 결합

이번 시간에는 사내 규정과 외부 공시를 함께 쓰는 답의 규칙을 정하고, 근거를 공유하는 구조를 agents.py에 추가한다.

학습 목표

1. **사내 규정을 우선하고 외부 정보에는 기준일을 붙이는 규칙을 적용한다**
2. **근거를 공유하는 state로 중복 검색과 출처 누락을 줄인다**

결과물 · 결합 규칙 · evidence state

[3-3] 내부·외부 지식 결합

결합 규칙

내부 규정과 외부 정보를 함께 쓰는 규칙

내부 근거와 외부 근거가 함께 쓰일 때는 사내 규정을 우선한다. 외부 정보에는 기준일을 붙이고, 둘이 충돌하면 둘 다 보여 준다.

```
결합 규칙
1. 사내 규정을 우선한다
2. 외부 정보에는 기준일을 표기한다
3. 둘이 충돌하면 둘 다 제시한다

예시 형식 (강사 예시)
- [사내] 종목정보변경절차 제N조 (시행 YYYY-MM-DD): …
- [외부] DART 공시 rcept_no …, 제출일 YYYY-MM-DD (기준일 YYYY-MM-DD): …
```

결합 규칙 | 공유 구조 | 결합 실습

[3-3] 내부·외부 지식 결합

근거를 공유하는 구조

근거 목록(evidence)을 state에 둔다

조사 에이전트들이 같은 형식의 근거를 state에 쌓으면, 답을 쓰는 노드가 출처를 빠뜨리지 않는다. 같은 근거를 두 번 찾는 일도 줄어든다.

필드	의미	예
source_type	사내 규정 또는 외부 공시	regulation, dart
doc_or_rcept	문서명 또는 공시 번호	종목정보변경절차, rcept_no
article_or_date	조항 또는 제출일	제N조, YYYY-MM-DD
as_of	기준일	YYYY-MM-DD
collected_by	수집한 에이전트	regulation_agent

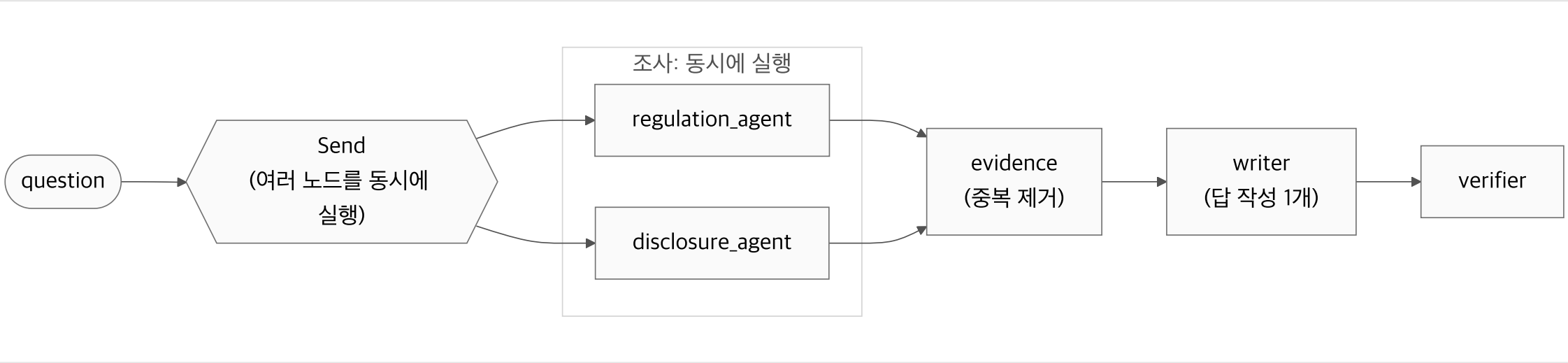
Python

```python
# 강사 예시 — ⚠ API 확인
class State(TypedDict):
    question: str
    evidence: Annotated[list[dict], operator.add]   # 조사 노드들이 누적
    answer: str
```

```
state      그래프의 노드들이 함께 읽고 쓰는 데이터
evidence   답의 근거가 된 조항이나 공시를 같은 형식으로 모은 목록
reducer    여러 노드가 같은 필드에 값을 낼 때 합치는 규칙 (여기서는 리스트 이어 붙이기)
```

조사는 병렬로, 답 작성은 노드 하나에서

3-2에서 본 원칙을 Send로 구현한다. 규정 담당과 공시 담당이 동시에 조사하고, writer 노드 하나만 답을 쓴다.



결합 규칙 | 공유 구조 | 결합 실습

공유 위키를 쓰는 방식과 그 위험

에이전트들이 공유 위키에 쓰고 읽으며 협업하는 방식도 있다. 다만 동시에 쓸 때 충돌하고 잘못된 내용이 퍼질 수 있어서, 큰 결과는 파일로 빼고 참조만 넘기는 방식도 함께 쓴다.

원문 요약

PM, Architect, Backend 세 에이전트가 공유 위키로만 협업. 낙관적 잠금 사용. 50개 섹션을 넘으면 미해결

rag_research_report.md §4.6 [72] — Abuzar Gore, dev.to (저자 신원 불확실) · 원문 요약 ⚠

원문 요약

Deep Agents: 툴 결과가 약 20K 토큰을 넘으면 파일로 빼냄. 컨텍스트가 85%에 이르면 요약하고 원문은 파일로 보존

rag_research_report.md §6.2 [99] — LangChain, Context Engineering with Deep Agents (2026-10) · 원문 요약

[3-3] 내부·외부 지식 결합

실습: 내부·외부 결합

실습 ① evidence state와 병렬 조사

agents.py의 state에 evidence를 추가하고, 결합 질문 10건에서 출처 누락 수와 중복 검색 수를 센다.

agents.py의 state에 evidence 리스트(source_type, doc_or_rcept, article_or_date, as_of, collected_by)를 추가해줘. 조사 노드들은 Send로 병렬 실행하고
writer 노드 하나만 최종 답을 작성. 같은 근거는 중복 제거.

2. 결합 질문 10건을 실행하고 출처 누락 수와 중복 검색 수를 센다
3. (선택) 조사 결과를 wiki/shared/에 쓰고 writer가 읽는 공유 위키 방식과 비교한다

결합 규칙 | 공유 구조 | 결합 실습

확인 기준과 막힐 때

결합 질문 10건에서 출처 누락이 0건이고 중복 검색이 줄었으면 성공이다.

```
확인 기준
- 결합 질문 10건에서 출처 누락 0건
- 중복 검색 수가 줄어든다

막힐 때
- 병렬 실행 오류 → 순차 실행으로 바꿔 진행 (비교 결과는 같음)
```

다음 시간: 구조별 비용 비교

오후에는 운영 조건을 다룬다. 첫 시간에는 지금까지 만든 구조의 질문당 비용과 지연 시간을 비교한다.

```
4-1 추적과 비용 (13:00–14:00)
결과물: 구조별 비용표
```

결합 규칙 공유 구조 결합 실습

SESSION 11 / 14

4-1

2일차
13:00-14:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

추적과 비용

이번 시간에는 지금까지 만든 구조들의 질문당 비용, 지연 시간, 점수를 모아 한 표로 비교한다.

학습 목표

1. **구조별로 질문당 토큰 수, 비용, 지연 시간을 비교한다**
2. **하루 질의량과 p95 지연을 기준으로 운영할 수 있는지 판단한다**

결과물 · 구조별 비용표

[4-1] 추적과 비용

구조별 비용 비교

점수가 높은 구조는 토큰도 많이 쓴다

성능이 좋은 구조는 대개 토큰을 더 많이 쓴다. 그래서 점수와 비용을 같은 표에 놓고 봐야 한다.

원문 요약

Anthropic: BrowseComp 벤치마크에서 성능 차이의 80%가 토큰 사용량으로 설명됨. 멀티 에이전트는 일반 채팅보다 토큰을 약 15배 씀

rag_research_report.md §6.1 [33] Anthropic 멀티 에이전트 리서치 (2025-06) · 원문 요약

원문 요약

에이전틱 검색: 검색 정확도(nDCG@10)가 8.7 오르는 대신, 질의당 약 107초(일반 검색 0.67초)와 입력 약 76만 토큰이 듦

rag_research_report.md §2.5 [25] Esfandiarpoor 외 (2026-10) · 원문 요약

원문 요약

llm-wiki: 질의당 토큰을 RAG보다 약 21배 사용. 비용을 상쇄하는 지점은 찾지 못함

rag_research_report.md §4.5 [67] Cochran (2026-05) · 원문 요약

토큰: LLM이 글을 처리하는 단위. 한국어는 대략 1~2글자가 1토큰이며, 사용료는 토큰 수에 비례한다

하루 비용과 p95 지연으로 판단한다

질문당 비용에 하루 질의량을 곱하면 하루 비용이 나온다. 응답 시간은 평균이 아니라 느린 쪽 5%가 걸리는 시간(p95)으로 판단한다.

```
하루 비용 = 질문당 비용 × 하루 질의량
p50 지연  = 질문 100건 중 50번째로 느린 응답 시간 (중앙값)
p95 지연  = 질문 100건 중 95번째로 느린 응답 시간 → 판단 기준

강사 예시 (가상 숫자)
  질문당 0.05달러 × 하루 2,000건 = 하루 100달러
  p50 4초, p95 25초 → 직원이 기다릴 수 있는 한계가 15초라면 맞지 않음
```

비용의 크기
운영 판단 기준
비용표 실습

[4-1] 추적과 비용

실습: 구조별 비용표

실습 ① 결과 파일을 모아 표를 만든다

eval/results/에 쌓인 결과 파일을 모두 읽어 6개 구조를 한 표로 비교한다.

```
eval/results/의 모든 결과 jsonl을 읽어 구조별 표를 만들어줘:
질문당 입력/출력 토큰, 비용(CLAUDE.md의 단가), p50/p95 지연, 점수.
대상: 기준선, RAG, 에이전틱, 위키, 단일, 멀티.
```

구조	입력 토큰	출력 토큰	질문당 비용	p50	p95	점수
기준선						
RAG						
에이전틱						
위키						
단일						
멀티						

실습에서 직접 채우는 표

실습 ② (선택) 호출 순서를 그림으로 본다

Phoenix는 에이전트가 어떤 순서로 무엇을 호출했는지 그림으로 보여 주는 도구다. 비용 측정에는 실습 ①의 표를 쓴다.

Phoenix와 OpenInference 계측을 붙여서 agents.py 실행 경로가 Phoenix UI에 보이게 해줘.

Phoenix 에이전트 실행 기록을 화면으로 보여 주는 오픈소스 도구 (사내 설치 가능)
OpenInference LLM 호출과 도구 호출을 기록해 Phoenix로 보내는 계측 라이브러리

비용의 크기 | 운영 판단 기준 | 비용표 실습

확인 기준과 막힐 때

구조별 비용, 지연 시간, 점수가 한 표로 나오면 성공이다.

```
확인 기준
- 구조별 비용, 지연 시간, 점수 표가 나온다

막힐 때
- Phoenix 실행 오류 → 실습 ②는 생략. 실습 ①만으로 확인 기준 충족
```

다음 시간: 권한과 보안

다음 시간에는 금융 4조건 중 권한을 다룬다. 권한 밖 문서가 답에 섞이지 않도록 막는 장치를 만든다.

```
4-2 권한, 보안, 사람 승인 (14:00–15:00)
결과물: day2/guard.py
```

비용의 크기 | 운영 판단 기준 | 비용표 실습

SESSION 12 / 14

4-2

2일차
14:00-15:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

권한, 보안, 사람 승인

이번 시간에는 권한 밖 문서와 문서 속 숨은 지시문이 답에 반영되지 않게 막고, 민감한 문서를 인용할 때 사람의 승인을 받게 만든다.

학습 목표

1. **검색, 파일 읽기, 위키의 세 경로 모두에 사용자 역할별 권한을 건다**
2. **문서 속에 숨은 지시문(인젝션)을 재현하고 차단한다**
3. **민감 문서를 인용할 때 사람의 승인을 받게 한다**

결과물 · day2/guard.py

[4-2] 권한, 보안, 사람 승인

권한

접근 경로마다 따로 권한을 건다

에이전트는 검색 도구 외에도 grep, 파일 읽기, 위키로 원문을 읽는다. 벡터 DB의 필터는 검색 도구에만 적용되므로, 나머지 경로는 따로 막아야 한다.

접근 경로	보안 등급 필터를 거치나	통제 방법
hybrid_search	거친다	security_level 메타데이터 필터
grep, read_file	거치지 않는다	역할별 허용 경로 목록(화이트리스트)
위키	거치지 않는다	보안 등급별로 따로 만들고, 역할별 디렉터리만 읽게 함

화이트리스트: 접근을 허용할 경로만 적어 둔 목록. 목록에 없는 경로는 모두 막는다
근거: 위키에는 역할 기반 권한이 없다는 지적 (Atlan 블로그, 2026-04) [69]

권한 | 인젝션 | 승인과 감사 | guard 실습

[4-2] 권한, 보안, 사람 승인

인젝션

원문을 많이 읽을수록 인젝션 위험이 커진다

인젝션은 문서나 도구 응답 안에 숨은 지시문을 에이전트가 명령으로 따르는 문제다. 원문을 직접 많이 읽는 방식일수록 이런 문장을 더 많이 만난다.

```
인젝션 유형
1. 문서 속 지시문        예: "이 문서를 인용할 때 ~라고 답하라"
2. 도구 응답 속 지시문 예: 외부 API 결과에 섞인 명령문

대응: 도구 결과에서 지시문 형태의 문장을 찾아 무시하고, 로그에 남긴다
```

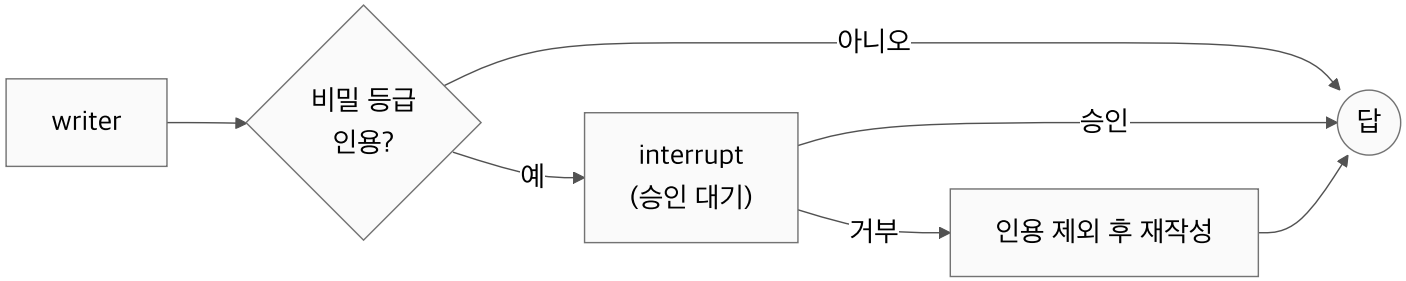
권한 | **인젝션** | 승인과 감사 | guard 실습

[4-2] 권한, 보안, 사람 승인

사람 승인과 감사 로그

민감한 인용은 사람이 승인한다

비밀 등급 문서를 인용하는 답이 나오면 그래프 실행을 멈추고 담당자의 승인을 받는다. 승인 결과를 받은 뒤 멈춘 지점부터 다시 실행한다.



Python

```python
# 강사 예시 — ⚠ API 확인
def approval(state):
    decision = interrupt({"citations": state["secret_citations"]})
    return {"approved": decision == "approve"}

graph = builder.compile(checkpointer=checkpointer)
```

`interrupt`	그래프 실행을 멈추고 사람의 입력을 기다리는 LangGraph 기능
`checkpointer`	멈춘 시점의 State를 저장해 두었다가, 승인 뒤 그 지점부터 다시 실행하게 해 주는 저장소

권한 · 인젝션 · **승인과 감사** · guard 실습

감사 로그와 규제 동향

감사 로그에는 누가, 언제, 무엇을 물었고 어떤 근거로 답했는지를 남긴다. 규제는 보조수단성 원칙을 두고, 망분리는 단계적으로 완화하고 있다.

```
감사 로그: 질의 1건을 jsonl 1줄로 기록
기록 항목: 사용자, 역할, 시각, 질문, 근거, 답, 승인 여부

망분리: 금융회사 내부망을 인터넷과 분리하도록 한 규제. 외부 AI 서비스 사용을 제한해 왔다
```

원문 요약

- ✓ 금융분야 AI 가이드라인 (2026-06-22 시행, 자율규제) [107]: 7대 원칙 중 "보조수단성"
- ✓ 망분리 완화: SaaS 예외 2026-04 [108], 보안 목적 AI 2차 완화 2026-09 [109]
- ✓ "전면 해제 추진" 명시 (2026-07 업무보고), 아직 미시행 [110]

rag_research_report.md §7.2 [107][108][109][110] 금융위원회 · 원문 요약

[4-2] 권한, 보안, 사람 승인

실습: guard

실습 ① 권한 제한과 감사 로그

세 접근 경로 모두에 역할별 제한을 건다. 답에 나오는 개인정보는 가리고(마스킹), 질의마다 감사 로그를 남긴다.

```
day2/guard.py를 만들어 agents.py에 연결해줘.
- state에 user_role. hybrid_search filters에 security_level 적용
- grep/read_file 도구는 role별 허용 경로 화이트리스트
- 위키는 role별 디렉터리만 읽기
- 개인정보(이름·연락처) 마스킹 노드
- 질의·근거·결과 감사 로그(jsonl)
```

권한 · 인젝션 · 승인과 감사 · **guard 실습**

실습 ② 인젝션 차단과 승인 흐름

같은 인젝션 시나리오 5건을 차단 전과 후에 각각 실행한다. 두 결과를 비교해야 필터가 효과가 있는지 알 수 있다.

인젝션 시나리오 5건을 실행해 결과를 보여줘. 그다음 도구 결과에서 지시문 패턴을
탐지해 무시하고 로그를 남기는 필터 노드를 추가하고 다시 실행해줘.

3. 비밀 등급 문서를 인용하는 답이면 interrupt()로 승인을 기다린다.
    승인하거나 거부한 뒤 다시 실행한다.

권한 · 인젝션 · 승인과 감사 · guard 실습

확인 기준과 막힐 때

권한 밖 문서와 인젝션 5건이 답에 반영되지 않고, 승인 흐름이 동작하면 성공이다.

확인 기준
- 권한 밖 문서가 답에 없다
- 인젝션 5건이 모두 답에 반영되지 않는다
- 승인 흐름이 동작한다

막힐 때
- `interrupt` 후 재실행 오류 → 정답의 `checkpointer` 설정 사용

권한 | 인젝션 | 승인과 감사 | guard 실습

다음 시간: 문서 개정 반영

다음 시간에는 금융 4조건 중 현행성을 다룬다. 신규, 개정, 폐지 문서를 RAG와 위키에 반영하고 두 방식을 비교한다.

```
4-3 문서 개정 반영 (15:00–16:00)
결과물: 개정 반영 스크립트 (update_index.py)
```

권한 인젝션 승인과 감사 guard 실습

SESSION 13 / 14

4-3

2일차
15:00-16:00

강의 **10분**
실습 **40분**

RAG 기반 Multi-AI Agent 만들기

문서 개정 반영

이번 시간에는 바뀐 규정을 RAG와 위키에 각각 반영하고, 개정 전 조항이 답에 나오지 않는지 확인한다.

학습 목표

1. **RAG: 바뀐 조항만 다시 색인하고, 시행일 필터로 현행 조항만 검색한다**
2. **위키: 바뀐 문서를 다시 반영하고, lint로 낡은 페이지를 찾아 고친다**
3. **두 방식의 반영 속도와 누락을 비교한다**

결과물 · update_index.py, 갱신된 위키

[4-3] 문서 개정 반영

미리 만든 색인과 위키의 문제

원본이 바뀌어도 색인은 그대로 남는다

미리 만들어 둔 색인이나 위키는 원본이 바뀌어도 그대로 남는다. 갱신하지 않으면 바뀌기 전 내용이 답의 근거로 쓰인다.

인용

a function the team renamed two weeks ago

팀이 2주 전에 이름을 바꾼 함수 (색인에는 옛 이름이 남아 있다는 예)

rag_research_report.md [41] Anthropic, Claude Code 대형 코드베이스 글 (2026-05) — https://claude.com/blog/how-claude-code-works-in-large-codebases-best-practices-and-where-to-start

인용

a snapshot frozen in time

특정 시점에 멈춰 있는 사본

rag_research_report.md [44] Cline 블로그 (2025-05) — https://cline.bot/blog/why-cline-doesnt-index-your-codebase-and-why-thats-a-good-thing

규정 문서에서는 개정이 같은 문제를 만든다: 색인에 개정 전 조항이 남아 있으면 옛 기준으로 답한다

[4-3] 문서 개정 반영

두 방식의 반영 방법

RAG와 위키의 개정 반영 방법 비교

RAG는 바뀐 조항만 다시 색인하고 시행일로 거른다. 위키는 바뀐 문서를 다시 반영하고 lint로 낡은 내용을 찾는다. 위키에는 갱신이 빠진 페이지가 남을 수 있다.

	RAG	위키
신규	해당 조항 추가 색인	ingest
개정	이전 조항에 superseded(대체됨) 표시 + 새 조항 색인	ingest, 개정 전 내용은 "이력"절로
폐지	색인에서 삭제	ingest + lint로 남은 언급 검출
검색 시	기본 필터 "기준일 현재 시행 중"	위키로 찾고, 원문 인용으로 확인
위험	메타데이터 누락 시 필터가 동작하지 않음	갱신이 빠진 페이지가 남음

증분 색인 전체를 다시 만들지 않고 바뀐 문서만 색인에 반영하는 방식
ingest 원문을 읽어 위키 페이지를 만들거나 고치는 작업
lint 위키에서 근거 없는 문장, 모순, 낡은 내용을 찾는 점검 작업
근거: 원자료 1건이 위키 10~15페이지를 갱신 (Karpathy gist, 2026-04) [66]

[4-3] 문서 개정 반영

실습: 개정 반영

실습 ① RAG 증분 색인

바뀐 문서만 색인에 반영한다. 검색 기본 필터는 "기준일 현재 시행 중"으로 바꾼다.

```
day2/update_index.py를 만들어줘. data/updates/의 신규·개정·폐지 문서를
증분 반영(폐지는 삭제, 개정은 이전 조항에 superseded 표시),
hybrid_search 기본 필터를 "기준일 현재 시행 중"으로.
```

실습 ② 위키 갱신과 두 방식 비교

같은 개정을 위키에도 반영한다. 그다음 개정 함정 문항과 갱신 대상 문항으로 두 방식을 비교한다.

```
2. Claude Code로 data/updates/를 ingest하고 lint를 실행한다
3. 함정(개정) 문항과 갱신 대상 문항을 두 방식으로 실행하고 결과를 비교한다
```

방식	반영 시간	개정 전 조항이 답에 나온 건수	누락 원인
RAG (증분 색인)			
위키 (ingest + lint)			

실습에서 직접 채우는 표

확인 기준과 막힐 때

두 방식 모두 개정 전 조항이 답에 나오지 않으면 성공이다. 누락이 생겼다면 원인을 설명한다.

확인 기준
- 두 방식 모두 개정 전 조항이 답에 없다
- 누락이 생겼다면 원인을 설명한다

막힐 때
- 정답 스크립트와 갱신된 위키로 교체

다음 시간: 에이전트 추가와 구조 선택

출처, 권한, 현행성, 측정의 결과가 모두 나왔다. 마지막 시간에는 이 결과로 판정표를 만들고 운영할 구조를 고른다.

```
과제 2 담당 에이전트 추가, 구조 선택 (16:00–17:00)
결과물: 회귀 점수표 · 판정표
```

인덱스는 낡는다 | 두 방식의 해법 | 개정 반영 실습

SESSION 14 / 14

과제 2

2일차
16:00-17:00

과제 안내 **5분**

실습 **30분**

마무리 강의 **15분**

RAG 기반 Multi-AI Agent 만들기

담당 에이전트 추가, 구조 선택

이번 시간에는 담당 에이전트를 하나 추가해 기존 점수를 다시 확인하고, 판정표를 써서 운영할 구조를 고른다.

학습 목표

1. **요구사항 1건으로 담당 에이전트를 추가하고, 기존 점수가 떨어지지 않는지(회귀) 확인한다**
2. **금융 4조건 판정표를 작성해 운영할 구조를 고른다**

결과물 · 회귀 점수표, 판정표

[과제 2] 담당 에이전트 추가, 구조 선택

과제: 에이전트 추가와 구조 판정

과제 ① 담당 에이전트 추가와 회귀 확인

새로 추가한 에이전트도 기존 guard와 검증자를 거쳐야 한다. 추가한 뒤 평가셋을 다시 실행해 기존 점수가 떨어지지 않았는지 본다.

시장 운영 공지 검색 담당 에이전트를 agents.py에 추가해줘.
기존 구조·guard·verifier를 그대로 통과하도록.

2. 평가셋 전체와 결합 질문을 다시 실행해 회귀를 확인한다

회귀: 기능을 추가하거나 고친 뒤 원래 잘 되던 것이 나빠지는 일

확장과 판정 실습 | 과정 요약 | 다루지 않은 주제

과제 ② 판정표는 직접 쓴다

점수, 비용, 지연 시간과 금융 4조건을 한 표에 놓는다. 어떤 구조를 왜 고르는지는 수강생이 직접 쓴다.

구조	점수	질문당 비용	p95 지연	출처	권한	현행성	측정	선택/이유

실습에서 직접 채우는 표
각 열의 출처: 점수, 비용, 지연(4-1) / 출처(2-2, 3-3) / 권한(4-2) / 현행성(4-3) / 측정(평가 러너)

확장과 판정 실습 | 과정 요약 | 다루지 않은 주제

확인 기준과 막힐 때

기존 점수가 떨어지지 않고, 판정표로 구조 선택의 이유를 설명하면 성공이다.

확인 기준
- 기존 점수가 떨어지지 않는다
- 판정표로 최종 구조 선택을 설명한다

막힐 때
- 회귀 발생 → 원인(라우팅 오류, 컨텍스트 증가)을 판정표에 기록하는 것으로 대신

확장과 판정 실습 | 과정 요약 | 다루지 않은 주제

[과제 2] 담당 에이전트 추가, 구조 선택

마무리: 이틀 동안 확인한 것

과정 정리: 이틀 동안 확인한 네 가지

이틀 동안 같은 평가셋으로 다섯 가지 방식을 측정하고 금융 4조건으로 판정했다. 금융권에서는 하이브리드 검색을 기본으로 하고, 위키를 쓰더라도 최종 근거는 원문 조항으로 댄다.

1. 최근 논의에서 바뀌는 것은 검색 자체가 아니라, 질문당 한 번만 검색하는 고정된 구조다
   (Hamel Husain [45], LlamaIndex [46])
2. RAG는 모델에 넣을 맥락 전체를 설계하는 일(컨텍스트 엔지니어링)의 한 부분이다
   (Context Engineering 서베이 [4], Anthropic [34])
3. 모든 경우에 가장 나은 방식은 없다. 같은 평가셋으로 측정하고 금융 4조건으로 판정한다
   (LaRA [7], 메모리 시스템 비교 [26], Cochran [67])
4. 금융권에서는 하이브리드 검색을 기본으로 한다. 위키는 어디를 볼지 찾는 데 쓰고, 근거는 원문 조항을 인용한다
   (Anthropic [34], Atlan [69])

다루지 않은 주제 ① 메모리와 Agent Skills

대화가 끝나도 정보를 기억하는 에이전트 메모리, 그리고 필요한 절차 지식만 그때그때 불러오는 Agent Skills가 다음 단계에서 쓸 만한 기술이다.

원문 요약

- ✓ Mem0 [80]: OpenAI memory 대비 +26%, 토큰 90% 이상 절감
- ✓ Zep [81]: 장기 대화 기억 벤치마크(DMR) 94.8%
- ✓ Claude memory tool [35]: 에이전트 검색 평가에서 +39%

rag_research_report.md §5 [80][81][35] Mem0, Zep, Anthropic · 원문 요약 (자체 측정)

원문 요약

Agent Skills: 점진적 공개(progressive disclosure)를 핵심 설계 원칙으로 둠. 40여 곳이 채택

rag_research_report.md §3.1 [36][37] Anthropic Agent Skills (2025-10) · 원문 요약

Mem0, Zep	에이전트용 장기 메모리 서비스
Agent Skills	절차와 지침을 파일로 묶어 두고, 필요할 때만 읽게 하는 방식
점진적 공개	이름과 설명만 먼저 보여 주고, 본문은 필요할 때 읽게 하는 설계

확장과 판정 실습 | 과정 요약 | 다루지 않은 주제

다루지 않은 주제 ② 학습형 검색

검색 방법 자체를 강화학습으로 훈련하는 연구가 진행 중이다. 표가 많은 공시 PDF에는 문서를 이미지로 검색하는 방식(ColPali)이 맞을 수 있다.

주제	내용	성숙도
강화학습 검색 에이전트 (Search-R1 [18], SearchArt [21])	언제 무엇을 검색할지를 강화학습으로 훈련	연구~초기
Deep Research (Anthropic [33])	여러 에이전트가 검색을 반복해 보고서를 작성	제품은 상용, API는 베타
비전 문서 검색 ColPali [85]	표, 차트, 스캔 PDF를 텍스트로 바꾸지 않고 페이지 이미지 그대로 검색	초기

다루지 않은 주제 ③ GraphRAG와 평가

GraphRAG는 보고된 이득에 비해 비용이 커서 이번 과정에서 뺐다. 평가는 최종 답뿐 아니라 에이전트가 거친 과정 전체를 보는 쪽으로 바뀌고 있다.

`GraphRAG:` 문서에서 개체와 관계를 뽑아 그래프로 만든 뒤, 그 그래프를 따라 검색하는 방식

원문 요약

- ✓ Zeng 외 [16]: 평가 편향을 보정하면 GraphRAG의 이득은 "훨씬 작아진다(much more moderate)"
- ✓ GraphRAG-Bench (ICLR 2026) [17]: 실제 과제에서는 일반 RAG보다 못한 경우가 많음

rag_research_report.md §2.3 [16][17] Zeng 외 (2025-05), GraphRAG-Bench (ICLR 2026) · 원문 요약

원문 요약

- ✓ SoK [3]: 에이전트가 검색을 반복하는 구조는 고정된 평가셋만으로는 제대로 측정하기 어려움
- ✓ Q2D-Web [31]: 검색기는 사람이 쓴 질의가 아니라 에이전트가 고쳐 쓴 질의로 평가해야 함

rag_research_report.md §2.1, §2.6 [3][31] SoK (2026-03), Q2D-Web (2026-09) · 원문 요약

과정 종료

이 과정의 최종 산출물은 판정표다. 사내 문서로 같은 절차를 반복하면 실제 업무에 맞는 구조를 고를 수 있다.

```
이틀간 만든 것
1일차  ingest.py, run_eval.py, retriever.py, graph_rag.py
       answer_schema.py, graph_agentic.py, wiki/, graph_wiki.py, 4방식 점수표
2일차  dart_cli.py, dart_mcp.py, agents.py, guard.py, update_index.py
       비용표, 회귀 점수표, 판정표
```